

Probabilistic Foundations for Project Management

Johan Jönsson

May 19, 2026

Contents

1	Introduction	3
1.1	Motivation	4
1.2	The Scope of the Report	5
2	Mathematical foundation for risk management	6
2.1	Project duration and work-package durations	6
2.2	Project cost and work-package cost without risk	9
2.3	Risks in Projects	10
2.3.1	Risk within a Work Package	12
2.3.2	Risk within the Whole Project	13
3	Simulation of Cost and Time Under Standard Assumptions	14
3.1	Model Assumptions for the Baseline Simulation	15
3.1.1	Baseline Assumptions for Work-Package Durations	15
3.1.2	Time-invariant cost assumptions	15
3.1.3	Risk-sampling assumptions	16
3.2	From Model Assumptions to Simulation Algorithms	17
3.2.1	Work-Package-Level Simulation of Duration and Cost	17
3.2.2	Calculating the Project Completion Time	19
3.2.3	Exact Completion-Time Distribution for a Small Network	21
3.3	Validation Benchmark: Exact Completion-Time Distribution for a Small Network	23
3.3.1	Shifted Poisson Duration Model and Grouped Exact CDF Algorithm . . .	23
3.3.2	Experimental Validation of the Monte Carlo Project-Duration Simulation	27
4	Practical Full Cost and Time Simulation with Risk	35
4.1	Purpose of the Full Project Example	35
4.2	Reference Project and Connection to Project Risk Practice	35
4.3	How the Data Is Translated into the Simulation Framework	36
4.4	Implemented Schedule and Cost Model	36
4.5	Expected-Value Checks for the Simulation	37
4.6	Risk-Level Diagnostics and Risk Prioritisation	39

4.7	Project-Level Duration and Cost Diagnostics	40
4.8	Activity-Level Cost and Schedule Exposure	41
4.8.1	Definition of the Cost and Critical-Time Exposure Metric	42
4.8.2	Practical Interpretation	44
A	Useful Theorems for Analytical Expected-Value Checks	47
B	Mathematical niceties	52

1 Introduction

The aim of this technical report is to develop a project management framework in which project time, cost, and risk can be described using a clear probabilistic structure. The mathematical part of the report is not introduced for its own sake, but to create a clean foundation on which a simulation framework can be built. By first defining the project in a general probabilistic way, the assumptions used in the later simulation model become easier to state, interpret, and modify.

This is important because project simulation often depends heavily on assumptions about activity durations, cost development, dependency structure, and risk occurrence. A general mathematical framework makes these assumptions explicit. It shows which parts of the model are structural features of the project, such as the precedence network, and which parts are modeling choices, such as independence assumptions, distributional assumptions, or the way risks are connected to work packages. In this sense, the framework helps clarify how general the model can be before additional assumptions are imposed for computation.

A central idea in this report is therefore that a project should not only be treated as a fixed deterministic plan, but as a system that evolves over time as uncertainty is resolved and new information becomes available. This makes it possible to describe project progress, cost, remaining work, and risk exposure in a more structured way. It also provides a basis for simulation methods that estimate project outcomes under uncertainty, while still making clear what assumptions are needed for the simulation to be valid.

The purpose of the report is thus both theoretical and practical. The theoretical framework gives a precise language for describing uncertain project behavior, while the simulation framework shows how this language can be used computationally to study project duration, cost, and risk effects. Together, these parts provide a foundation for more systematic project risk management and for future extensions to decision-making during project execution.

The report also introduces an activity-level exposure metric that connects cost and schedule information in one diagnostic plot. This is motivated by the fact that traditional project-risk tools often separate these perspectives: probability-impact matrices rank risks qualitatively, while common schedule-risk metrics mainly focus on how activities affect project duration.¹ Recent quantitative risk-prioritisation work also emphasizes that risks can have different impacts depending on whether the objective is project duration or project cost.² The metric introduced here asks which work packages matter for both cost control and schedule control by comparing each activity's share of total project cost with its share of project completion time when it is critical. It should be understood as a screening metric, not a causal sensitivity measure, but it gives project managers a simple way to identify activities where cost exposure and schedule exposure overlap.

¹M. Vanhoucke. *Schedule Risk Analysis: Measuring the Time Sensitivity of an Activity*. 2011. URL: https://www.pmknowledgecenter.com/dynamic_scheduling/risk/schedule-risk-analysis-measuring-time-sensitivity-activity (visited on 05/19/2026).

²F. Acebes et al. "Beyond probability-impact matrices in project risk management: A quantitative methodology for risk prioritisation". In: *Humanities and Social Sciences Communications* 11.670 (2024). DOI: 10.1057/s41599-024-03180-5. URL: <https://doi.org/10.1057/s41599-024-03180-5>.

1.1 Motivation

Stochastic process theory has been used successfully in several fields where decisions must be made under uncertainty. In financial mathematics, stochastic processes are used to model the evolution of financial variables in risk management, portfolio management, and derivatives pricing.³ This provides an important example of how a rigorous probabilistic framework can support both theoretical modeling and practical decision-making.

A similar connection exists in modern artificial intelligence, especially in reinforcement learning. Reinforcement learning is built on sequential decision-making under uncertainty, often formalized through Markov decision processes and related stochastic models.⁴ This is closely related to project management, where the state of a project changes over time and decisions must be updated as new information becomes available.

There is already research applying stochastic processes and reinforcement learning ideas to project management. For example, stochastic project management has been proposed as a way to describe project organization, planning, resources, and managerial actions under uncertainty.⁵ More recent work has also applied Markov chains and Markov decision processes to integrate project risk analysis with risk response planning.⁶ These studies suggest that project management can benefit from a more formal probabilistic description of project evolution.

However, the mathematical foundation used to describe the project itself is still not as systematically developed as in fields such as financial mathematics or reinforcement learning. Therefore, if project management is to make fuller use of modern simulation, data-driven decision support, and AI-based methods, it is useful to reformulate its foundations in terms of stochastic process theory. This report takes a step in that direction by introducing a discrete-time probabilistic framework for project management and risk management.

This report is also motivated by recent attempts to make project risk analysis more quantitative and simulation-based. Acebes et al. propose a Monte Carlo approach for prioritising project risks by modelling their probability and impact on project duration and cost, rather than relying only on probability-impact matrices.⁷ Bara similarly develops an enhanced Monte Carlo framework in which risks can affect both cost and schedule, occur at different points in time, and influence later risks through dependency relationships.⁸

These works show that project risk management is moving toward richer computational models. However, they also highlight the need for a clearer mathematical language connecting

³A. Meucci. “Review of Discrete and Continuous Processes in Finance: Theory and Applications”. In: *SSRN Electronic Journal* (2009). DOI: 10.2139/ssrn.1373102. URL: <https://ssrn.com/abstract=1373102>.

⁴T. Lattimore and C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020. ISBN: 9781108486828. DOI: 10.1017/9781108571401.

⁵N. Oberemok and I. Oberemok. “Stochastic Processes in Project Management”. In: *EUREKA: Social and Humanities* 6 (2018), pp. 3–9. DOI: 10.21303/2504-5571.2018.00799.

⁶S. Soltan and M. Ashrafi. “Application of Reinforcement Learning for Integrating Project Risk Analysis and Risk Response Planning: A Case Study on Construction Projects”. In: *Journal of Project Management* 10.1 (2025), pp. 71–86. DOI: 10.5267/j.jpm.2024.11.001.

⁷Acebes et al., see n. 2.

⁸M. Bara. “Enhanced Monte Carlo Simulation for Project Risk Analysis: Integrating Cost and Schedule Impacts with Time-Shifted Risks and Dependency Modeling”. In: *Applied Operations and Analytics* 1.1 (2025), pp. 1–15. DOI: 10.1080/29966892.2025.2552675.

the project-management concepts, the probabilistic assumptions, and the simulation algorithm. The contribution of this report is therefore not only to build another simulation tool, but to formulate the project itself in a way that makes the assumptions transparent. By defining work packages, durations, costs, risks, and information over time in one framework, it becomes easier to see which assumptions are essential, which are only used for computation, and how the model can be generalized.

This is especially important if such methods are to become useful for practitioners. Many commercial tools can perform advanced project risk simulation, but their internal assumptions and algorithms are often difficult to inspect or modify. The framework developed here is more limited than mature commercial software, but it is designed to be transparent, reproducible, and extendable. Since the implementation can be inspected, cloned, and modified, it provides a practical starting point for building more advanced project-management simulation tools while keeping a clear connection between the mathematical model and the software implementation.

1.2 The Scope of the Report

The scope of this report is to develop both a mathematical framework and a simulation framework for project management under uncertainty. The first part introduces the notation needed to describe a project probabilistically. This includes work packages, activity durations, start and finish times, costs, risks, and the information available as the project evolves over time. The purpose of this mathematical part is not only to make the model more rigorous, but also to make the assumptions behind the later simulation framework explicit.

This is important because every simulation model depends on assumptions. For example, one must decide how activity durations are distributed, whether risks are independent, how risks are connected to work packages, and how cost depends on time. By first writing the project model in a general mathematical form, it becomes clearer which assumptions are part of the general framework and which assumptions are added later only to make the model computationally usable. In this way, the mathematical framework provides a clean basis for understanding how general the simulation model can be.

The second part of the report describes how this framework can be implemented computationally using Monte Carlo simulation. The attached Python code shows how uncertain activity durations, project networks, costs, and risk effects can be simulated in a structured way. The report therefore connects the mathematical definitions to the actual algorithms used in the simulation framework. This makes the approach understandable both from a theoretical perspective and from a software/project-management perspective.

The report also includes validation of the network simulation method. For a small project network, a non-Monte Carlo calculation is used as a benchmark. This makes it possible to compare the simulated completion-time distribution with an exact or directly computed reference result. This validation is important because it checks not only that the code runs, but also that the project-network logic gives the expected probabilistic output.

For the larger integrated cost and schedule simulation, the report uses project activity and risk

data based on the case study in⁹. This makes it possible to test the framework on a more realistic project structure with both duration and cost effects. In this larger setting, exact distributional validation is no longer practical, so the diagnostics instead focus on simulation behavior, sample estimates, and quantities that can be checked analytically, such as expected values for certain metrics. Comparing analytic expected values with simulated sample means gives a useful diagnostic for whether the implemented risk and cost models behave as intended.

The report should therefore be understood as a foundation and prototype rather than a complete commercial project-risk tool. Its main contribution is to show how project-management quantities can be described in one coherent probabilistic framework, and how this framework can then be used to build transparent and extendable simulation methods for project time, cost, and risk.

2 Mathematical foundation for risk management

This section introduces the basic probabilistic language used to describe a project under uncertainty. The goal is not to make the project model unnecessarily abstract, but to give a clear structure for discussing how a project can unfold over time. The project is assumed to already have a schedule, work packages, durations, costs, and identified risks. The question is then how these quantities behave when the future is uncertain.

For a project manager, the main idea is simple. A project does not have only one possible outcome. Instead, it has many possible ways of developing, depending on activity durations, cost changes, risk events, and new information that becomes available during execution. We represent the collection of all possible project outcomes by Ω .

As the project progresses, more information becomes available. We write $\mathcal{F}_t$ for the information available up to time t . Thus, $\mathcal{F}_t$ represents what is known about the project at that point in time, such as which activities have started or finished, which risks have occurred, and what costs have been observed. The collection $(\mathcal{F}_t)_{t \geq 0}$ describes how project information grows over time.

Finally, the probability measure $\mathbb{P}$ assigns probabilities to the possible project events. Together, these objects form the filtered probability space

$$(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t \geq 0}, \mathbb{P}).$$

In this report, this framework is mainly used as a clean way to organize assumptions. It makes clear what is being simulated, what information is available at each time, and how uncertainty in duration, cost, and risk is connected to the project schedule.

2.1 Project duration and work-package durations

The lowest planning unit in a project schedule is the work package. In this report, work packages are therefore the basic units used to define project duration. We assume that each work package

⁹Acebes et al., see n. 2.

has a unique number and that the project contains N work packages. The set of work packages is written as

$$\mathcal{W} = \{1, 2, \dots, N\}.$$

To describe the schedule logic, we use the predecessor operator $\text{Pred}(i)$. For a work package $i \in \mathcal{W}$, the set $\text{Pred}(i)$ contains the immediate predecessors of i , meaning the work packages that must be completed before work package i can begin.

The simplest case is a project with two work packages, where work package 2 can only start after work package 1 has finished.



Figure 1: Simple finish-start dependency between two work packages.

We define the duration of work package i as the random variable D_i . We also define its start time by S_i and its finish time by T_i . If work package i has one immediate predecessor, then its start time is the finish time of that predecessor:

$$S_i = T_{\text{Pred}(i)}.$$

The finish time of work package i is then

$$T_i = S_i + D_i.$$

Some work packages have no predecessors and can therefore start at the beginning of the project. We define the initial set as

$$\mathcal{I} = \{i \in \mathcal{W} : \text{Pred}(i) = \emptyset\}.$$

For all $i \in \mathcal{I}$, we have

$$S_i = 0,$$

and hence

$$T_i = D_i.$$

The next important case is when several work packages can run in parallel. In the example below, work packages 1 and 2 can both start at time 0.

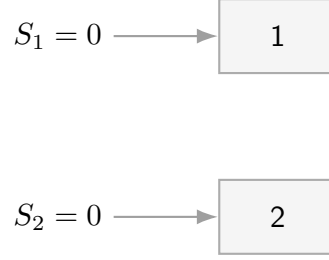


Figure 2: Two work packages that can start at the beginning of the project.

We also need to handle merge activities. A merge activity is an activity with more than one immediate predecessor. If work package i depends on several predecessors, then it cannot start until all of them have finished. Therefore, its start time is determined by the latest finishing predecessor:

$$S_i = \max_{j \in \text{Pred}(i)} T_j.$$

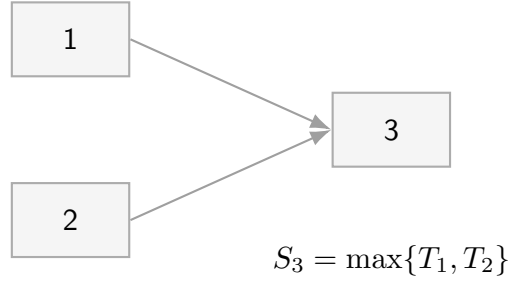


Figure 3: A merge activity starts only after all immediate predecessors have finished.

In many project schedules, a dependency can also include a lag. A lag means that even after predecessor j has finished, activity i may need to wait an additional amount of time before starting. We denote this lag by $L_{j,i}$. With lags included, the start time becomes

$$S_i = \max_{j \in \text{Pred}(i)} (T_j + L_{j,i}).$$

When there is no lag, we simply have $L_{j,i} = 0$.

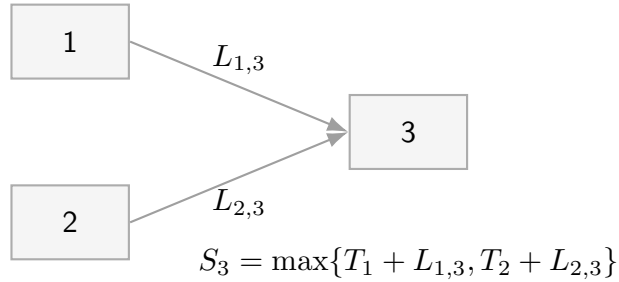


Figure 4: A merge activity with dependency lags.

Definition 2.1. A work package i is described by its start time S_i and duration D_i . If i has

no predecessors, then

$$S_i = 0.$$

If i has at least one predecessor, then

$$S_i = \max_{j \in \text{Pred}(i)} (T_j + L_{j,i}),$$

where $L_{j,i}$ is the lag on the dependency from work package j to work package i . The finish time of work package i is then

$$T_i = S_i + D_i.$$

Once the durations and lags are known, the finish time of every work package can be calculated from the project network. The total project completion time is the latest finish time among all work packages:

$$T_{\text{Project}} = \max_{i \in \mathcal{W}} T_i.$$

2.2 Project cost and work-package cost without risk

We have now defined how the work packages are related in time. The next step is to describe their cost. Each work package may generate cost while it is active, and the total project cost is obtained by adding these work-package costs together with any project-level costs.

For a fixed time interval $n < t \leq m$, we write the cost of work package i over this active interval as

$$C_i((t)_{n < t \leq m}).$$

This notation means that the cost is allowed to depend on the whole time interval during which the work package is active, not only on the two endpoints n and m . This is useful because cost may depend on daily labor cost, resource usage, calendar time, seasonal effects, or other quantities that vary during the active period.

For example, in a simple additive daily-cost model, one could have

$$C_i((t)_{n < t \leq m}) = \sum_{t=n+1}^m K_i(t),$$

where $K_i(t)$ is the cost generated by work package i at time t . However, the notation also allows more general cost models where the total cost over the interval is not just a simple sum.

To keep the notation readable, we will from now on use the shorthand

$$C_i(n, m) = C_i((t)_{n < t \leq m}).$$

Thus, whenever we write $C_i(n, m)$, it should be understood as the cost of work package i over the whole active interval $n < t \leq m$.

Since work package i is active during the random interval $S_i < t \leq T_i$, its realized cost is written

as

$$C_i(S_i, T_i).$$

Using the shorthand above, this means

$$C_i(S_i, T_i) = C_i((t)_{S_i < t \leq T_i}).$$

The total cost that can be directly assigned to work packages is therefore

$$C_{WP} = \sum_{i \in \mathcal{W}} C_i(S_i, T_i).$$

Here, C_{WP} only includes costs attached to specific work packages.

Some costs may instead belong to the project as a whole. Examples include general overhead, administration, management cost, or other costs that are not naturally assigned to one specific work package. We denote these additional project-level costs by O . The exact definition of O should be adapted to the project being studied.

The total project cost without explicit risk effects is then

$$C_{Project} = C_{WP} + O.$$

The following result shows how the expected project cost can be decomposed by conditioning on the possible start and finish times of each work package.

Theorem 2.1. *Assume that the cost of each work package has finite expected value and that $\mathbb{E}[|O|] < \infty$. Then*

$$\mathbb{E}[C_{Project}] = \sum_{i \in \mathcal{W}} \sum_{m > n} \mathbb{P}(T_i = m, S_i = n) \mathbb{E}[C_i(n, m) \mid T_i = m, S_i = n] + \mathbb{E}[O].$$

This formula says that the expected project cost can be calculated by considering all possible active intervals for each work package. For each possible interval, we multiply the probability of that interval by the expected cost of the work package on that interval, and then sum over all work packages. Finally, the expected project-level cost $\mathbb{E}[O]$ is added separately.

2.3 Risks in Projects

The two main project quantities discussed so far are duration and cost. These quantities describe the baseline project model. In project risk management, we also need to describe additional uncertain effects that may change the project outcome relative to this baseline.

In this report, risk is understood as the combination of the probability of a consequence and the magnitude or severity of that consequence.¹⁰ A risk is therefore not only defined by whether

¹⁰Society for Risk Analysis. *Society for Risk Analysis Glossary*. Updated August 2018. Expert group led by Terje Aven. Society for Risk Analysis. Aug. 2018. URL: <https://www.sra.org/wp-content/uploads/2020/04/SRA-Glossary-FINAL.pdf>, p. 4.

something can happen, but also by how large its effect is if it happens. This includes both negative effects, such as delays, rework, equipment failure, or cost overruns, and positive effects, such as cost reductions or shorter activity durations.

The distinction between baseline uncertainty and risk effects is important. For example, uncertainty in the ordinary duration of a work package belongs to the baseline model, since the work package itself is assumed to be carried out. A risk effect is instead treated as an additional consequence relative to this baseline, such as an equipment failure that delays the work package or an unexpected opportunity that reduces cost.

This distinction is not always sharp in practice. A work package taking longer than planned can sometimes be modeled as ordinary duration uncertainty and sometimes as a realized delay risk. The choice depends on the purpose of the model. In this report, we separate the baseline model from risk variables by treating a risk variable as an additional project effect measured relative to the baseline.

Definition 2.2. A risk-effect variable R is a random variable representing an additional project consequence relative to the baseline model. The value $R = 0$ means that the risk has no realized effect, while $R \neq 0$ means that the risk has a realized effect on the project. We require

$$\mathbb{P}(R \neq 0) > 0.$$

The risk associated with R is described by both the probability that $R \neq 0$ and the magnitude of the realized consequence.

2.3.1 Risk within a Work Package

A work-package risk is a risk effect that is attributed to one specific work package. This is the most natural form of risk in the simulation framework, because the effect can be connected directly to the activity where it occurs.

For work package i , we distinguish between schedule risks and cost risks. A schedule risk changes the duration of the work package, while a cost risk changes the cost generated by the work package.

Definition 2.3. A schedule risk attached to work package i is denoted by

$$R_i^{t,j},$$

where t indicates that it is a time effect, j is the index of the risk, and i is the work package to which the risk is attached.

A cost risk attached to work package i is denoted by

$$R_i^{c,j}((t)_{S_i < t \leq T_i}),$$

where c indicates that it is a cost effect. The notation means that the cost risk may depend on the time interval during which work package i is active.

Let Z_i denote the baseline duration of work package i , before schedule-risk effects are added. If work package i has schedule risks indexed by j , then the realized duration is modeled as

$$D_i = Z_i + \sum_j R_i^{t,j}.$$

Thus, the final duration of the work package consists of its baseline duration plus the realized schedule-risk effects.

Similarly, the realized cost of work package i can be written as its baseline cost plus realized cost-risk effects:

$$C_i((t)_{S_i < t \leq T_i}) = C_i^0((t)_{S_i < t \leq T_i}) + \sum_j R_i^{c,j}((t)_{S_i < t \leq T_i}),$$

where C_i^0 denotes the baseline cost of work package i without explicit risk effects.

2.3.2 Risk within the Whole Project

Some risks are not naturally assigned to one specific work package. These are project-level risks. Examples include general management disruption, project-wide inflation effects, contractual penalties, or external events that affect the project as a whole.

A project-level cost risk can be added to the project-level cost term O . If the risk affects cost over the full active project interval, it can be written as

$$R^{c,j}((t)_{0 < t \leq T_{Project}}).$$

The total project-level cost term may then include such effects.

Project-level schedule risks are more delicate. If a schedule risk is not attached to a specific work package, it is not always clear where in the network the delay should be added. Adding it only at the end of the project may be useful as a simple approximation, but it does not show how the risk affects the critical path or the timing of later activities. For this reason, schedule risks should, whenever possible, be attached to the work package or dependency where they actually affect the project network.

3 Simulation of Cost and Time Under Standard Assumptions

The previous section described project time, cost, and risk under a general probabilistic framework. In this section, we introduce a simpler set of assumptions that are common in project Monte Carlo simulation. These assumptions are not required by Monte Carlo simulation itself. A Monte Carlo simulation only requires that each simulated scenario is sampled from the joint probability model that has been specified. However, in many project-management applications, this joint model is not written explicitly. As a result, activity durations, risk occurrences, and risk impacts are often sampled independently unless a dependence mechanism is added.

The purpose of this section is therefore to make these standard assumptions explicit. This gives a clear baseline model for simulation, and it also shows which assumptions must be changed if one wants to include correlations, conditional risk dependencies, Bayesian networks, copulas, dynamic probability updates, or other more advanced dependence structures. In this sense, the section describes a standard starting point for project Monte Carlo simulation, rather than the most general possible model.

Recent project-risk simulation work shows why a clearer probabilistic foundation is useful. Acebes et al. move beyond qualitative probability-impact matrices by using Monte Carlo simulation to quantify how individual risks affect project duration and project cost.¹¹ Bara develops this direction further by adding time-shifted risks and dependency rules, so that the occurrence of one risk can change the timing and probability of later risks.¹²

These contributions show that project risk management is moving toward richer simulation-based methods. However, they are mainly formulated from a project-management and algorithmic perspective. The underlying joint probability model is usually not written as a separate mathematical object, and assumptions such as independent sampling, conditional dependence, or dependence induced by the project network are therefore not always clearly separated.

The purpose of the framework in this report is to make these assumptions explicit. By first describing the project using the common language of probability theory, we can state exactly when durations, costs, risk occurrences, and risk impacts are assumed to be independent, and when a more general dependence structure is needed. The independent-sampling model developed in this section should therefore be understood as a standard baseline for Monte Carlo project simulation. It is not the most general possible model, but it is a transparent starting point from which more advanced models, such as correlated sampling, conditional risk dependencies, or time-shifted risk effects, can be added in a controlled way.

¹¹Acebes et al., see n. 2.

¹²Bara, see n. 8.

3.1 Model Assumptions for the Baseline Simulation

3.1.1 Baseline Assumptions for Work-Package Durations

We first assume that the baseline durations of the work packages are mutually independent. Let Z_i denote the baseline duration of work package i , before schedule-risk effects are added. The independence assumption means that for any values $n_1, \dots, n_N$,

$$\mathbb{P}(Z_1 = n_1, \dots, Z_N = n_N) = \prod_{i=1}^N \mathbb{P}(Z_i = n_i).$$

This is stronger than pairwise independence and is the assumption usually used when each work-package duration is sampled separately in a Monte Carlo simulation.

If work package i has schedule-risk effects $R_i^{t,j}$, then its realized duration is modeled as

$$D_i = Z_i + \sum_j R_i^{t,j}.$$

Thus, the final duration consists of the baseline duration plus the realized time effects assigned to that work package.

3.1.2 Time-invariant cost assumptions

We also assume that the cost of a work package is time-invariant. This means that the distribution of the cost depends on how long the work package is active, but not on the calendar time at which it is performed. Formally, for any shift k , we assume

$$C_i((t)_{S_i < t \leq T_i}) \stackrel{d}{=} C_i((t)_{S_i+k < t \leq T_i+k}).$$

Under this assumption, shifting the work package forward or backward in time does not change the distribution of its cost.

Therefore, when the active interval has length n , we use the shorthand

$$C_i(n) = C_i((t)_{0 < t \leq n}).$$

This does not mean that the cost is deterministic or linear with respect to duration. It only means that, under the standard assumption in this section, the distribution of the work-package cost depends on the length of the active interval rather than on its absolute calendar position.

This assumption is important to state explicitly. In many project Monte Carlo models, activity costs are sampled from fixed cost parameters, or from cost parameters linked to activity duration, without making the calendar date of the activity part of the model. Such models therefore use a time-invariance assumption, even when it is not stated directly. Under this assumption, the same work package has the same cost distribution whether it is performed early or late in the project, as long as its duration is the same.

By writing this assumption explicitly, we make clear what is being excluded from the baseline

model. Calendar-dependent effects, such as inflation, seasonal labor availability, procurement timing, weather windows, or changes in resource prices, are not captured unless they are added as separate model components. The time-invariant model should therefore be understood as a standard simplifying assumption for baseline simulation, not as a necessary property of project cost.

The total project cost without project-level risk effects can then be written as

$$C_{Project} = \sum_{i \in \mathcal{W}} C_i(D_i) + O,$$

where O denotes project-level costs that are not assigned to a specific work package.

Theorem 3.1. *Assume that each work-package cost has finite first moment and that $\mathbb{E}[|O|] < \infty$. Under the time-invariant cost assumption,*

$$\mathbb{E}[C_{Project}] = \sum_{i \in \mathcal{W}} \sum_{n=1}^{\infty} \mathbb{P}(D_i = n) \mathbb{E}[C_i(n) \mid D_i = n] + \mathbb{E}[O].$$

This formula is useful because it does not require every work-package cost to be independent of every other work-package cost. For the expected total cost, we only need to add the expected contribution from each work package and then add the expected project-level cost. This makes the formula practical even when some costs are related in reality.

Independence becomes more important when we want to go beyond the expected cost. For example, if we want to estimate the full range of possible total project costs, calculate cost percentiles, or study worst-case cost outcomes, then we need to be clear about how costs across different work packages are related. If no such relationship is specified, the simulation will typically treat them as independently sampled.

3.1.3 Risk-sampling assumptions

We now state the corresponding simplified assumptions for risk effects. In this baseline simulation model, each risk is either assigned to one specific work package or treated as a project-level risk. We do not include risks that directly affect several work packages at once. This restriction keeps the simulation model simple and makes it clear where each risk effect enters the project network.

For a schedule risk assigned to work package i , we write

$$R_i^{t,j}.$$

For a cost risk assigned to work package i , we write

$$R_i^{c,j}((t)_{S_i < t \leq T_i}).$$

Under the time-invariance assumption, the distribution of the cost-risk effect does not depend

on the absolute calendar position of the work package. Thus, for any shift k ,

$$R_i^{c,j}((t)_{S_i < t \leq T_i}) \stackrel{d}{=} R_i^{c,j}((t)_{S_i+k < t \leq T_i+k}).$$

When convenient, this allows us to write the cost-risk effect as depending on the active duration rather than the calendar interval. We also assume that for this Monte Carlo implementation, risk occurrences and risk severities are sampled independently across work packages and risks, unless a dependence rule is explicitly added.

Under these assumptions, the simulation proceeds by sampling baseline durations, risk occurrences, risk severities, and costs, then propagating the resulting durations through the fixed project network. This gives a transparent baseline model for project duration, project cost, critical paths, and risk effects. More advanced models can be obtained by replacing the independent-sampling assumptions with explicit dependence structures.

3.2 From Model Assumptions to Simulation Algorithms

We now translate the assumptions from the previous section into the simulation algorithms used in the implementation. The goal is to show how the model is computed in practice, first at the work-package level and then at the project-network level. This makes clear how the sampled durations, costs, and risk effects are combined before the full project completion time and total project cost are calculated.

3.2.1 Work-Package-Level Simulation of Duration and Cost

We now describe how the simplified assumptions above are implemented in the Monte Carlo simulation. The main idea is that each Monte Carlo replication represents one possible realization of the project. Instead of simulating one project outcome at a time, the code stores all simulated outcomes in vectors. This makes the implementation easier to read and also makes the connection to the mathematical notation clearer.

Let M denote the number of Monte Carlo replications. For work package i , we write the simulated duration vector as

$$\mathbf{D}_i = \begin{bmatrix} D_i^{(1)} \\ D_i^{(2)} \\ \vdots \\ D_i^{(M-1)} \\ D_i^{(M)} \end{bmatrix}.$$

Here, $D_i^{(r)}$ is the realized duration of work package i in Monte Carlo replication r . Under the standard assumptions in this section, these replications are sampled independently from the same distribution for work package i .

The duration of work package i is generated by first sampling the baseline duration Z_i and then adding the schedule-risk effects assigned to that work package. In vectorized form this is

written as

$$\mathbf{D}_i = \mathbf{Z}_i + \sum_j \mathbf{R}_i^{t,j}.$$

Thus, each entry of $\mathbf{D}_i$ gives the realized duration of work package i in one simulated project scenario.

We next simulate the cost generated while the work package is active. Under the time-invariant assumption, the cost distribution depends on the length of the active interval, but not on the absolute calendar time at which the work package is performed. A flexible way to model this is to assign a random daily cost to each active day. Let $K_{i,t}^{(r)}$ denote the daily cost of work package i on active day t in replication r . Then the accumulated daily cost in replication r is

$$A_i^{(r)} = \sum_{t=1}^{D_i^{(r)}} K_{i,t}^{(r)}.$$

In vector form, this becomes

$$\mathbf{A}_i = \begin{bmatrix} \sum_{t=1}^{D_i^{(1)}} K_{i,t}^{(1)} \\ \sum_{t=1}^{D_i^{(2)}} K_{i,t}^{(2)} \\ \vdots \\ \sum_{t=1}^{D_i^{(M-1)}} K_{i,t}^{(M-1)} \\ \sum_{t=1}^{D_i^{(M)}} K_{i,t}^{(M)} \end{bmatrix}.$$

This is the simulated version of the cost term $C_i(D_i)$ introduced above. It allows the cost to vary from day to day inside the work package, while still keeping the model time-invariant because the distribution depends on active duration rather than calendar date.

The following pseudocode gives the vectorized implementation of this daily-cost model.

Algorithm 1 Simulating accumulated daily cost $\mathbf{A}_i$ for work package i

- 1: **Input:** work package i , duration vector $\mathbf{D}_i$, number of replications M
 - 2: Initialize $\mathbf{A}_i \leftarrow \mathbf{0}$ ▷ Accumulated daily cost
 - 3: **for** $t = 1, \dots, \max_r D_i^{(r)}$ **do**
 - 4: Generate daily cost vector $\mathbf{K}_{i,t} \in \mathbb{R}^M$
 - 5: Define active mask $\mathbf{s}_t = \mathbf{1}_{\{\mathbf{D}_i \geq t\}}$
 - 6: $\mathbf{A}_i \leftarrow \mathbf{A}_i + \mathbf{K}_{i,t} \odot \mathbf{s}_t$
 - 7: **end for**
 - 8: **Output:** $\mathbf{A}_i$
-

The mask $\mathbf{s}_t$ keeps track of which simulated work packages are still active on active day t . If $D_i^{(r)} \geq t$, then replication r is still active and the daily cost is added. If $D_i^{(r)} < t$, then the work package has already finished in that replication and no additional daily cost is added.

A simpler version is obtained when each replication has one random daily cost rate that remains constant while the work package is active. In that case, we generate one cost-rate vector $\mathbf{K}_i$

and calculate

$$\mathbf{A}_i = \mathbf{K}_i \odot \mathbf{D}_i.$$

Algorithm 2 Simulating accumulated cost with a constant daily cost rate

- 1: **Input:** work package i , duration vector $\mathbf{D}_i$, number of replications M
 - 2: Generate daily cost-rate vector $\mathbf{K}_i \in \mathbb{R}^M$
 - 3: $\mathbf{A}_i \leftarrow \mathbf{K}_i \odot \mathbf{D}_i$
 - 4: **Output:** $\mathbf{A}_i$
-

The first algorithm allows the daily cost to vary from active day to active day. The second algorithm assumes that the daily cost rate is constant within each realization of the work package. Both are compatible with the time-invariant assumption, because neither model makes the cost distribution depend on the absolute calendar position of the work package.

Finally, the total simulated cost of work package i is obtained by adding the accumulated daily cost, any additional non-daily cost terms, and the realized cost-risk effects. If H_i^ℓ denotes an additional baseline cost component and $R_i^{c,j}$ denotes a cost-risk effect, then in vectorized form we write

$$\mathbf{C}_i = \mathbf{A}_i + \sum_{\ell} \mathbf{H}_i^{\ell} + \sum_j \mathbf{R}_i^{c,j}.$$

At a high level, the simulation for each work package is therefore structured as follows:

1. Generate the realized duration vector $\mathbf{D}_i$ by sampling the baseline duration $\mathbf{Z}_i$ and adding the schedule-risk effects $\mathbf{R}_i^{t,j}$.
2. Use the realized duration vector $\mathbf{D}_i$ to calculate the accumulated daily cost $\mathbf{A}_i$.
3. Add any additional cost components $\mathbf{H}_i^{\ell}$ and cost-risk effects $\mathbf{R}_i^{c,j}$ to obtain the total work-package cost vector $\mathbf{C}_i$.

This procedure is repeated for each work package. After all work-package duration and cost vectors have been generated, the project network is used to calculate project start times, finish times, total project duration, total project cost, and other project-level quantities across all Monte Carlo replications.

3.2.2 Calculating the Project Completion Time

We now describe how the project completion time is calculated efficiently from the project network. The schedule is assumed to be fixed and non-random, while the work-package durations are random variables. Since each start time and finish time is defined from sums and maxima of random variables, the output of the network calculation is also a random variable. Thus, the same recursion can be understood mathematically as defining S_i , T_i , and T_{Project} as random variables.

In the Monte Carlo implementation, this recursion is evaluated samplewise. Each work-package duration is represented by a vector of simulated values, where each entry corresponds to one

simulated project scenario. For each scenario, the sampled durations are treated as realized values, and the network calculation returns the corresponding project completion time and one selected critical path. Collecting these outputs over all scenarios gives a simulated sample from the distribution of T_{Project} .

The assumption that the schedule is a directed acyclic graph is natural for a standard project schedule. It means that the precedence structure cannot contain loops. In practical terms, a work package cannot depend, directly or indirectly, on itself, since that would make the schedule impossible to execute.

For implementation details on longest path calculations in directed acyclic graphs, see¹³. In the Python implementation, the topological order is obtained using the NetworkX topological sort function.¹⁴

Algorithm 3 Project completion time and one critical path in a project network

```

1: Input: Topological order  $\mathcal{T}$ , durations  $D_1, \dots, D_N$ , lags  $L_{j,i}$ 
2: Initialize  $S_i \leftarrow 0$  for all  $i \in \mathcal{W}$   $\triangleright S_i$  stores the earliest start time of activity  $i$ 
3: Initialize  $T_i \leftarrow 0$  for all  $i \in \mathcal{W}$   $\triangleright T_i$  stores the earliest finish time of activity  $i$ 
4: Initialize  $p_i \leftarrow \emptyset$  for all  $i \in \mathcal{W}$   $\triangleright p_i$  stores one predecessor on a critical path
5: for  $i \in \mathcal{T}$  do
6:   if  $\text{Pred}(i) = \emptyset$  then
7:      $S_i \leftarrow 0$ 
8:      $T_i \leftarrow D_i$ 
9:      $p_i \leftarrow \emptyset$ 
10:  else
11:     $m \leftarrow \arg \max_{j \in \text{Pred}(i)} (T_j + L_{j,i})$ 
12:     $S_i \leftarrow T_m + L_{m,i}$ 
13:     $T_i \leftarrow S_i + D_i$ 
14:     $p_i \leftarrow m$ 
15:  end if
16: end for
17:  $r \leftarrow \arg \max_{i \in \mathcal{W}} T_i$ 
18:  $T_{\text{Project}} \leftarrow T_r$ 
19:  $\mathcal{C} \leftarrow []$ 
20: while  $r \neq \emptyset$  do
21:   Add  $r$  to the front of  $\mathcal{C}$ 
22:    $r \leftarrow p_r$ 
23: end while
24: Output: Project completion time  $T_{\text{Project}}$  and one critical path  $\mathcal{C}$ 

```

This algorithm gives the project completion time for one simulated scenario. In the implementation, the same logic is applied in vectorized form across all Monte Carlo replications. Thus, instead of storing one value T_i for each work package, the code stores a vector of finish times,

¹³D. S. Hochbaum. *Lecture Notes for IEOR 266: Graph Algorithms and Network Flows*. Lecture notes, Department of Industrial Engineering and Operations Research, University of California, Berkeley. Updated 2024. Accessed: 2026-05-12. 2024. URL: <https://hochbaum.ieor.berkeley.edu/files/266Notes-F2020.pdf>.

¹⁴NetworkX Developers. *topological.sort*. NetworkX 3.6.1 documentation. Accessed: 2026-05-12. 2025. URL: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.dag.topological_sort.html.

one for each simulated project scenario.

The algorithm also returns one critical path. If there are no ties, this is the unique critical path. If there are ties, the $\arg \max$ operation selects one of the tied predecessors. This is sufficient for calculating the project completion time, since all tied predecessors give the same finish time. However, it is not sufficient if the goal is to count all critical paths or estimate the probability that a work package belongs to at least one critical path. For that purpose, the implementation must keep track of all predecessors that attain the maximum, not only one of them.

3.2.3 Exact Completion-Time Distribution for a Small Network

The Monte Carlo method estimates the distribution of T_{Project} by repeated simulation. For small discrete project networks, it is also possible to calculate the distribution exactly. This gives a useful validation benchmark for the simulation code.

For this exact calculation, assume that there are no lags and that the work-package durations $D_1, \dots, D_N$ are independent nonnegative integer-valued random variables. Let $\mathcal{P}$ be the set of complete paths through the project network. For a path $p_\theta \in \mathcal{P}$, define its duration by

$$D(p_\theta) = \sum_{i \in p_\theta} D_i.$$

Then the project completion time is

$$T_{\text{Project}} = \max_{p_\theta \in \mathcal{P}} D(p_\theta).$$

The difficulty is that different paths may share work packages. Therefore, the path durations are generally not independent, even when the individual work-package durations are independent. To handle this, we condition on the durations of the work packages that are shared by more than one path.

Let

$$\mathcal{N} = \{i \in \mathcal{W} : \text{work package } i \text{ belongs to at least two paths in } \mathcal{P}\}$$

be the set of shared work packages. For each path p_θ , define the non-shared part of the path by

$$\mathcal{U}_\theta = p_\theta \setminus \mathcal{N}.$$

After conditioning on the durations of the shared work packages, the remaining path sums are independent because they depend on disjoint sets of work-package durations.

Theorem 3.2. *Assume that the work-package durations $D_1, \dots, D_N$ are independent nonnegative integer-valued random variables and that the project network has no lags. Then, for every $t \geq 0$,*

$$\mathbb{P}(T_{\text{Project}} \leq t) = \sum_{(d_i)_{i \in \mathcal{N}} \in \mathbb{N}_0^{|\mathcal{N}|}} \left(\prod_{i \in \mathcal{N}} \mathbb{P}(D_i = d_i) \right) \prod_{p_\theta \in \mathcal{P}} \mathbb{P} \left(\sum_{k \in \mathcal{U}_\theta} D_k \leq t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i \right).$$

For computation at a fixed value of t , the sum can be restricted to

$$\mathcal{D}_t(\mathcal{N}) = \{0, \dots, t\}^{|\mathcal{N}|},$$

because all durations are nonnegative. If any shared duration exceeds t , then no complete path containing that work package can have total duration at most t .

Algorithm 4 Exact calculation of $\mathbb{P}(T_{\text{Project}} \leq t)$ for a small discrete project network

```

1: Input: Duration distributions  $\mathbb{P}_{D_1}, \dots, \mathbb{P}_{D_N}$ , threshold  $t$ , shared set  $\mathcal{N}$ , path set  $\mathcal{P}$ 
2:  $P \leftarrow 0$ 
3: for  $d = (d_i)_{i \in \mathcal{N}} \in \mathcal{D}_t(\mathcal{N})$  do
4:    $P_d \leftarrow 1$ 
5:   for  $i \in \mathcal{N}$  do
6:      $P_d \leftarrow P_d \cdot \mathbb{P}(D_i = d_i)$ 
7:   end for
8:   for  $p_\theta \in \mathcal{P}$  do
9:      $r_\theta \leftarrow t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i$ 
10:    if  $r_\theta < 0$  then
11:       $P_d \leftarrow 0$ 
12:      break
13:    else
14:       $P_d \leftarrow P_d \cdot \mathbb{P}\left(\sum_{k \in \mathcal{U}_\theta} D_k \leq r_\theta\right)$ 
15:    end if
16:  end for
17:   $P \leftarrow P + P_d$ 
18: end for
19: Output:  $P$ 

```

This exact method is useful for validation, but it does not scale well. If the quantities

$$\mathbb{P}\left(\sum_{k \in \mathcal{U}_\theta} D_k \leq r_\theta\right)$$

have already been precomputed and can be evaluated in constant time, then the algorithm scales as

$$O\left((t+1)^{|\mathcal{N}|}(|\mathcal{N}| + |\mathcal{P}|)\right).$$

The exponential dependence on $|\mathcal{N}|$ comes from summing over all possible duration combinations for the shared work packages:

$$d \in \{0, \dots, t\}^{|\mathcal{N}|}.$$

This poor scaling is not only a weakness of this particular algorithm. It is consistent with known complexity results for stochastic project networks. Hagstrom showed that, for PERT networks with independent task durations having discrete finite ranges, computing a value of the cumulative distribution function of the project duration is #P-complete, and computing the

mean is at least as hard.¹⁵ Since such a PERT network is a special case of the type of discrete project network considered here, this suggests that an exact polynomial-time method for the general completion-time distribution should not be expected.

This is why the exact method is mainly useful for small networks. For larger project networks, Monte Carlo simulation is more practical, while the exact calculation remains valuable as a benchmark for testing whether the simulation logic is correct.

3.3 Validation Benchmark: Exact Completion-Time Distribution for a Small Network

The previous subsection showed how the project completion time can be computed from the project network and how this calculation is used inside the Monte Carlo simulation. To check that the simulation behaves correctly, we now introduce a small discrete validation example where the completion-time distribution can also be computed by an exact non-Monte Carlo method.

The purpose of this example is not to claim that the shifted Poisson distribution is the best duration model for all project-management applications. Instead, it is used because it gives a simple integer-valued duration model with useful computational properties. This makes it possible to compare the Monte Carlo estimate of the project completion-time distribution with an exact benchmark.

3.3.1 Shifted Poisson Duration Model and Grouped Exact CDF Algorithm

For the validation example, we use a shifted Poisson model for the work-package durations. This model is mainly used for prototyping and exact validation, because it gives a simple discrete duration model with useful computational properties.

We assume that

$$D_i - a_i \sim \text{Poi}(\lambda_i),$$

where $a_i \in \mathbb{N}_0$ is the minimum duration and $\lambda_i > 0$. Thus,

$$\mathbb{E}[D_i] = a_i + \lambda_i.$$

We write this as

$$D_i \sim \text{ShiftedPoi}(\lambda_i, a_i).$$

The shifted Poisson model has two useful properties for this validation example. First, it allows durations to be larger than any fixed upper bound, while still assigning very small probability to very large values. This avoids assuming too early that extreme delays are impossible. Second, it is closed under summation. If the durations are independent and

$$D_i \sim \text{ShiftedPoi}(\lambda_i, a_i),$$

¹⁵hagstrom1988computational.

then for any index set $\mathcal{I}$,

$$\sum_{i \in \mathcal{I}} D_i \sim \text{ShiftedPoi} \left(\sum_{i \in \mathcal{I}} \lambda_i, \sum_{i \in \mathcal{I}} a_i \right).$$

This means that path sums can be represented by another shifted Poisson random variable, instead of being computed through repeated convolutions.

Let

$$\mathcal{P} = \{p_1, \dots, p_q\}$$

be the set of complete paths through the project network. For each work package i , define its path-incidence set by

$$S_i = \{\theta \in \{1, \dots, q\} : i \in p_\theta\}.$$

Thus, S_i records exactly which complete paths contain work package i .

For every nonempty subset

$$S \subseteq \{1, \dots, q\},$$

define the group of work packages with the same path-incidence pattern by

$$\mathcal{I}_S = \{i : S_i = S\}.$$

For this group, define

$$\Lambda_S = \sum_{i \in \mathcal{I}_S} \lambda_i, \quad A_S = \sum_{i \in \mathcal{I}_S} a_i.$$

If $\mathcal{I}_S = \emptyset$, these sums are interpreted as zero.

For each group, we then have

$$\sum_{i \in \mathcal{I}_S} D_i = A_S + Z_S,$$

where

$$Z_S \sim \text{Poi}(\Lambda_S).$$

The random variables $(Z_S)_S$ are independent, because they are sums over disjoint groups of independent work-package durations.

For each path p_θ , define its deterministic minimum duration by

$$A_\theta = \sum_{i \in p_\theta} a_i.$$

Then the duration of path p_θ can be written as

$$D(p_\theta) = A_\theta + \sum_{S: \theta \in S} Z_S.$$

The important computational point is that groups with $|S| = 1$ only affect one path. They

do not create dependence between different path durations. Therefore, they do not need to be explicitly enumerated. They can instead be handled directly using the Poisson CDF for the corresponding path.

Only the groups with $|S| \geq 2$ create dependence between several paths. Define

$$\mathcal{G} = \{S \subseteq \{1, \dots, q\} : |S| \geq 2, \Lambda_S > 0\}.$$

This is the set of non-singleton path-incidence groups that must be explicitly summed over.

For a fixed threshold $t \in \mathbb{N}_0$, define the remaining slack for path p_θ by

$$b_\theta(t) = t - A_\theta.$$

If $b_\theta(t) < 0$ for at least one path, then that path cannot finish by time t , even at its minimum duration. Therefore,

$$\mathbb{P}(T_{\text{Project}} \leq t) = 0.$$

Otherwise, for each $S \in \mathcal{G}$, define

$$u_S(t) = \min_{\theta \in S} b_\theta(t).$$

This is a valid upper bound because if $Z_S > u_S(t)$, then at least one path containing the group S must exceed the threshold t .

Theorem 3.3. *Assume that the work-package durations are independent and satisfy*

$$D_i \sim \text{ShiftedPoi}(\lambda_i, a_i).$$

Let $\mathcal{P} = \{p_1, \dots, p_q\}$ be the set of complete paths, and define the path-incidence groups as above. Let

$$\mathcal{G} = \{S \subseteq \{1, \dots, q\} : |S| \geq 2, \Lambda_S > 0\}.$$

For each path p_θ , define

$$A_\theta = \sum_{i \in p_\theta} a_i, \quad b_\theta(t) = t - A_\theta.$$

If $b_\theta(t) < 0$ for at least one θ , then

$$\mathbb{P}(T_{\text{Project}} \leq t) = 0.$$

Otherwise,

$$\mathbb{P}(T_{\text{Project}} \leq t) = \sum_{(z_S)_{S \in \mathcal{G}} \in \prod_{S \in \mathcal{G}} \{0, \dots, u_S(t)\}} \left(\prod_{S \in \mathcal{G}} \mathbb{P}(Z_S = z_S) \right) \prod_{\theta=1}^q F_{\text{Poi}(\Lambda_{\{\theta\}})} \left(b_\theta(t) - \sum_{\substack{S \in \mathcal{G} \\ \theta \in S}} z_S \right),$$

where

$$Z_S \sim \text{Poi}(\Lambda_S),$$

and

$$F_{\text{Poi}(\Lambda_{\{\theta\}})}(x) = \mathbb{P}(\text{Poi}(\Lambda_{\{\theta\}}) \leq x),$$

with the convention that

$$F_{\text{Poi}(\Lambda_{\{\theta\}})}(x) = 0 \quad \text{for } x < 0.$$

This formula is more efficient than summing over all shared work packages individually. The general exact formula enumerates all possible duration combinations for the shared work packages. Under the shifted Poisson assumption, work packages with the same path-incidence pattern can be grouped together. The summation is then performed only over the non-singleton incidence groups in $\mathcal{G}$.

Algorithm 5 Exact calculation of $\mathbb{P}(T_{\text{Project}} \leq t)$ under shifted Poisson durations

```

1: Input: Parameters  $(\lambda_i, a_i)_{i=1}^N$ , threshold  $t$ , complete paths  $\mathcal{P} = \{p_1, \dots, p_q\}$ 
2: for  $i = 1, \dots, N$  do
3:    $S_i \leftarrow \{\theta \in \{1, \dots, q\} : i \in p_\theta\}$ 
4: end for
5: for each nonempty incidence set  $S$  do
6:    $\Lambda_S \leftarrow \sum_{i:S_i=S} \lambda_i$ 
7:    $A_S \leftarrow \sum_{i:S_i=S} a_i$ 
8: end for
9: for  $\theta = 1, \dots, q$  do
10:   $A_\theta \leftarrow \sum_{i \in p_\theta} a_i$ 
11:   $b_\theta \leftarrow t - A_\theta$ 
12:  if  $b_\theta < 0$  then
13:    Output: 0
14:  end if
15: end for
16:  $\mathcal{G} \leftarrow \{S : |S| \geq 2, \Lambda_S > 0\}$ 
17: for  $S \in \mathcal{G}$  do
18:   $u_S \leftarrow \min_{\theta \in S} b_\theta$ 
19: end for
20:  $P \leftarrow 0$ 
21: for  $z = (z_S)_{S \in \mathcal{G}} \in \prod_{S \in \mathcal{G}} \{0, \dots, u_S\}$  do
22:   $P_z \leftarrow 1$ 
23:  for  $S \in \mathcal{G}$  do
24:     $P_z \leftarrow P_z \cdot \mathbb{P}(\text{Poi}(\Lambda_S) = z_S)$ 
25:  end for
26:  for  $\theta = 1, \dots, q$  do
27:     $r_\theta \leftarrow b_\theta - \sum_{\substack{S \in \mathcal{G} \\ \theta \in S}} z_S$ 
28:     $P_z \leftarrow P_z \cdot F_{\text{Poi}(\Lambda_{\{\theta\}})}(r_\theta)$ 
29:  end for
30:   $P \leftarrow P + P_z$ 
31: end for
32: Output:  $P$ 

```

The scaling of the grouped shifted Poisson algorithm is controlled by

$$\prod_{S \in \mathcal{G}} (u_S(t) + 1),$$

rather than by

$$(t + 1)^{|\mathcal{N}|},$$

where $\mathcal{N}$ is the set of shared work packages in the general exact formula.

If all Poisson probability mass functions and cumulative distribution functions are precomputed and can be evaluated in constant time, then the main loop scales as

$$O \left(\left[\prod_{S \in \mathcal{G}} (u_S(t) + 1) \right] \left(|\mathcal{G}| + \sum_{S \in \mathcal{G}} |S| + |\mathcal{P}| \right) \right).$$

Since $u_S(t) \leq t$, the worst-case bound is

$$O \left((t + 1)^{|\mathcal{G}|} \left(|\mathcal{G}| + \sum_{S \in \mathcal{G}} |S| + |\mathcal{P}| \right) \right).$$

The improvement is that the exponent depends on the number of distinct non-singleton path-incidence groups, not directly on the number of shared work packages. In many prototype networks, several shared work packages have the same path-incidence pattern, so

$$|\mathcal{G}| \ll |\mathcal{N}|.$$

The shifted Poisson model therefore gives a useful exact benchmark for small and medium-sized validation examples, while Monte Carlo simulation remains the more practical method for larger project networks.

3.3.2 Experimental Validation of the Monte Carlo Project-Duration Simulation

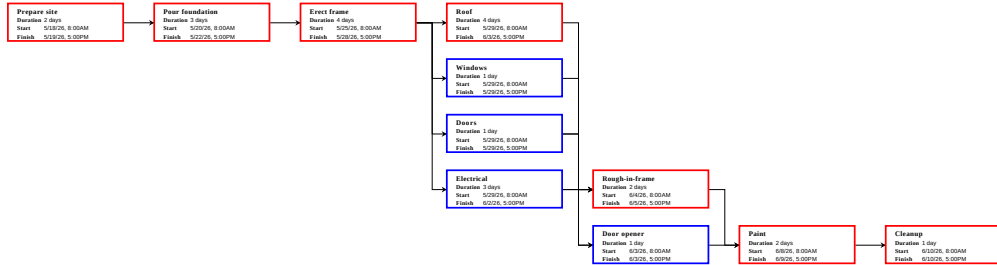


Figure 5: Garage project schedule used as the baseline project network. Only the upper part of the exported PDF is shown.

The validation experiment is based on the garage project schedule shown in Figure 5. The schedule defines the project network, meaning the work packages and their precedence relations.

In the experiment, this network structure is kept fixed, while the activity durations are treated as random variables.

The purpose of this experiment is to check whether the Monte Carlo simulation produces the same completion-time distribution as the exact discrete calculation from the previous subsection. The example is therefore not meant to be a full practical project-risk model. Instead, it is used as a controlled validation case where the simulation output can be compared against a non-Monte Carlo benchmark.

Each work package i is assigned an independent shifted-Poisson duration

$$D_i = a_i + X_i, \quad X_i \sim \text{Poi}(\lambda_i),$$

so that

$$\mathbb{E}[D_i] = a_i + \lambda_i.$$

Here, a_i represents the minimum possible duration of work package i , while λ_i controls the stochastic variation above this minimum. Thus, the deterministic garage schedule is used to define the precedence structure, and the shifted-Poisson variables are used to generate uncertain activity durations.

For each Monte Carlo replication, one duration is sampled for every work package. The project-network recursion is then applied to these sampled durations to obtain one simulated value of the project completion time T_{Project} . Repeating this procedure gives a Monte Carlo sample from the completion-time distribution. This simulated distribution is then compared with the exact CDF calculated using the grouped shifted-Poisson method.

Table 1: Shifted-Poisson duration assumptions used in the Monte Carlo experiment.

i	Work package	Pred(i)	a_i	λ_i	Modes	$\mathbb{E}[D_i]$
1	Prepare site	$\emptyset$	1	2	2, 3	3
2	Pour foundation	1	1	3	3, 4	4
3	Erect frame	2	2	3	4, 5	5
4	Roof	3	1	4	4, 5	5
5	Windows	3	1	1	1, 2	2
6	Doors	3	1	1	1, 2	2
7	Electrical	3	1	3	3, 4	4
8	Rough-in-frame	4, 5, 6, 7	1	2	2, 3	3
9	Door opener	6, 7	1	1	1, 2	2
10	Paint	8, 9	1	2	2, 3	3
11	Cleanup	10	1	1	1, 2	2

Using the duration assumptions in Table 1, each Monte Carlo replication samples one duration for every work package. For replication s , this gives a realized duration vector

$$(D_1^{(s)}, D_2^{(s)}, \dots, D_N^{(s)}).$$

The project-network recursion is then evaluated for that replication. In this validation example,

no lags are included, so the start time of work package i is

$$S_i^{(s)} = \begin{cases} 0, & \text{Pred}(i) = \emptyset, \\ \max_{j \in \text{Pred}(i)} T_j^{(s)}, & \text{Pred}(i) \neq \emptyset. \end{cases}$$

The finish time is then

$$T_i^{(s)} = S_i^{(s)} + D_i^{(s)}.$$

Thus, the simulated project completion time in replication s is

$$T_{\text{Project}}^{(s)} = \max_{i \in \mathcal{W}} T_i^{(s)}.$$

The purpose of the experiment is to compare the Monte Carlo approximation with the exact shifted-Poisson CDF calculation from the previous subsection. Since the duration variables are integer-valued, the comparison is made on the integer grid

$$t = 0, 1, \dots, 40.$$

The exact completion-time CDF is

$$F(t) = \mathbb{P}(T_{\text{Project}} \leq t),$$

while the empirical Monte Carlo CDF is

$$\hat{F}_M(t) = \frac{1}{M} \sum_{s=1}^M \mathbf{1}_{\{T_{\text{Project}}^{(s)} \leq t\}},$$

where M is the number of Monte Carlo replications. We also compare the exact and empirical probability mass functions, where

$$p(t) = \mathbb{P}(T_{\text{Project}} = t).$$

The diagnostics below measure how closely the Monte Carlo simulation matches the exact distribution. The main quantities reported are the maximum absolute CDF error,

$$\max_t |\hat{F}_M(t) - F(t)|,$$

the mean absolute CDF error, the CDF root mean squared error, the PMF error, the survival-tail error, and the runtime comparison between the exact CDF calculation and the vectorized Monte Carlo simulation.

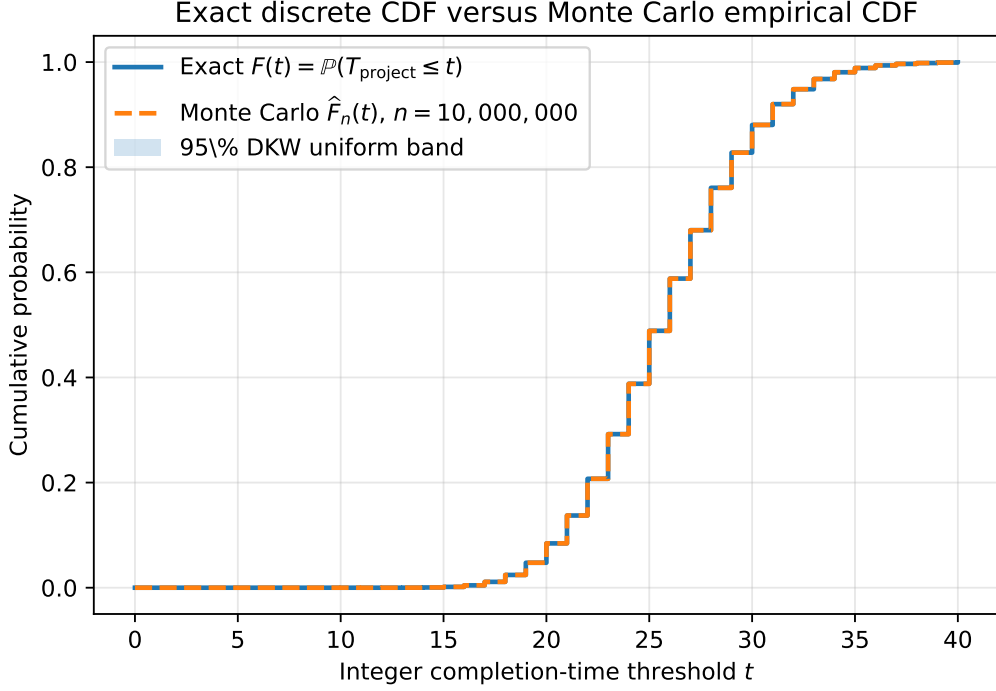


Figure 6: Exact discrete CDF compared with the Monte Carlo empirical CDF. The close agreement between the two curves shows that the Monte Carlo simulation reproduces the exact shifted-Poisson completion-time distribution on the evaluated integer grid.

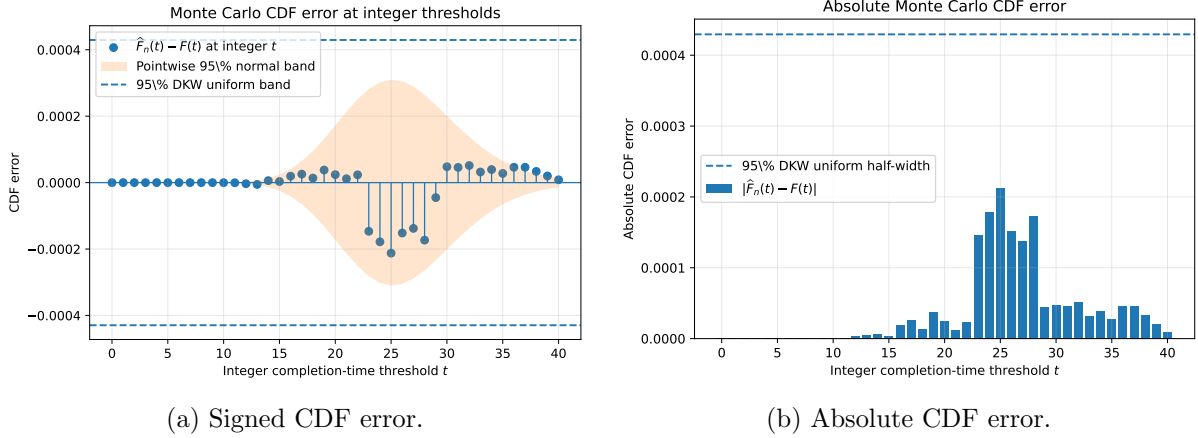
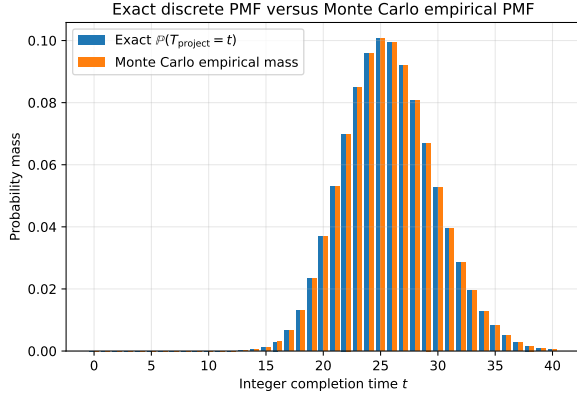
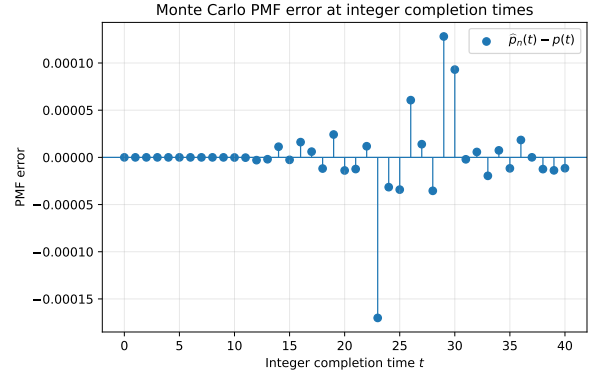


Figure 7: Monte Carlo CDF error diagnostics at integer completion-time thresholds. The dashed DKW band gives a uniform reference bound for the empirical CDF error, and the observed errors remain small across the evaluated grid.



(a) Exact and empirical PMF.



(b) PMF error.

Figure 8: Probability mass function diagnostics for the discrete project completion time. The empirical Monte Carlo probabilities closely match the exact shifted-Poisson probabilities at each integer completion time.

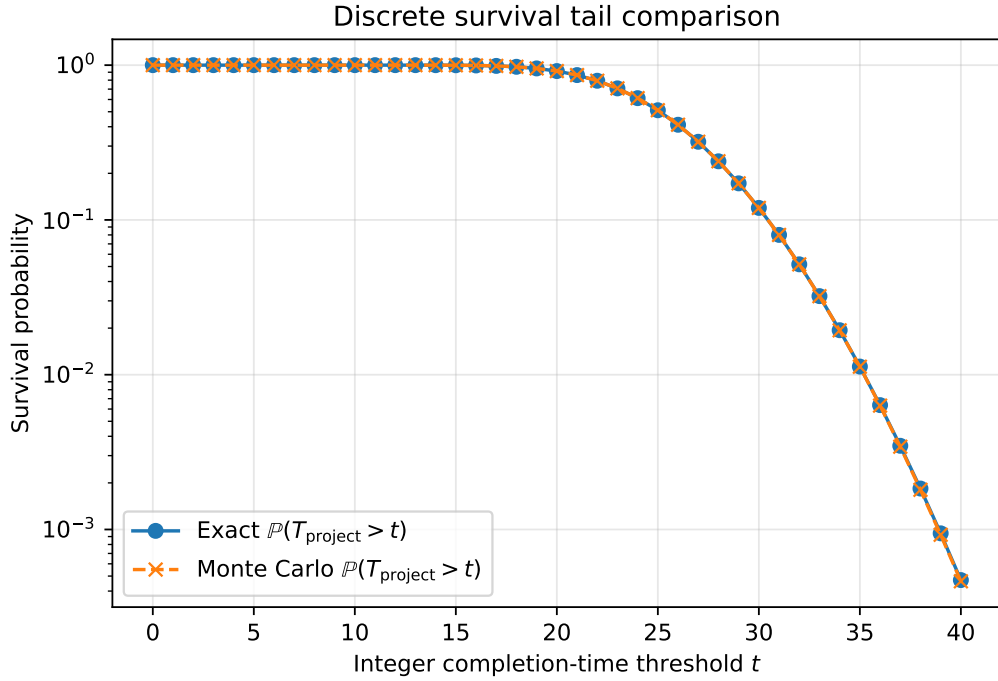
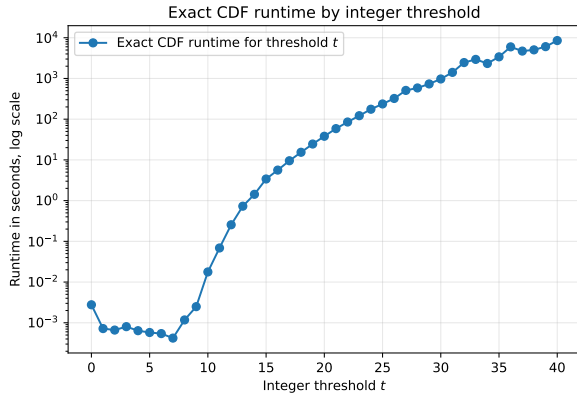


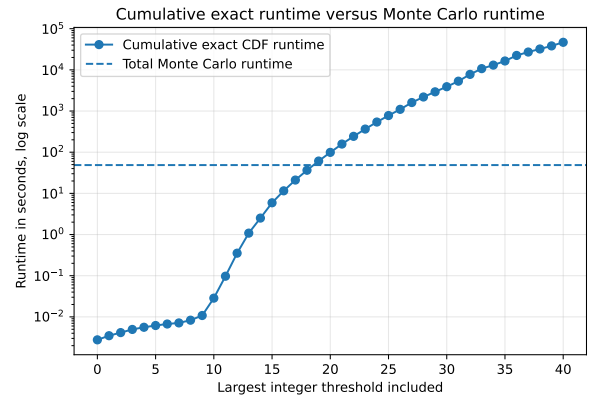
Figure 9: Survival-tail comparison between the exact distribution and the Monte Carlo estimate. The logarithmic scale makes small tail probabilities easier to inspect, which is important when assessing schedule-overrun risk.

Table 2: Monte Carlo validation diagnostics against the exact shifted-Poisson distribution.

Metric	Value
Monte Carlo samples n	10,000,000
Monte Carlo seed	2026
Monte Carlo runtime, seconds	48.6692
Exact CDF runtime, seconds	46720.2
Runtime ratio, exact / MC	959.953
Exact CDF grid	$t = 0, \dots, 40$
KS distance $\sup_t \hat{F}_n(t) - F(t) $	0.000212389
Integer threshold attaining max CDF error	$t = 25$
CDF MAE on integer grid	3.94731e-05
CDF RMSE on integer grid	6.85587e-05
95% DKW half-width	0.000429469
KS distance / DKW half-width	0.494538
PMF MAE on integer grid	1.91493e-05
PMF RMSE on integer grid	3.96243e-05
PMF total variation on grid	0.000392562
Exact tail mass $\mathbb{P}(T_{\text{project}} > 40)$	0.000471107
MC tail mass $\mathbb{P}(T_{\text{project}} > 40)$	0.0004626
Exact PMF mass on grid	0.999529
Max error between cumulative exact PMF and exact CDF	0



(a) Exact runtime by threshold.



(b) Cumulative exact runtime versus Monte Carlo runtime.

Figure 10: Runtime comparison between the exact CDF calculation and the Monte Carlo simulation. The exact method evaluates the CDF threshold by threshold, while the vectorized Monte Carlo method estimates the full completion-time distribution from one simulation run.

Table 4: Exact and Monte Carlo probability masses at the most likely integer completion times.

t	Exact $p(t)$	MC $\hat{p}_n(t)$	Error	Absolute error
25	0.100778	0.100744	-3.41723e-05	3.41723e-05
26	0.0993199	0.0993805	6.06375e-05	6.06375e-05
24	0.0958148	0.0957832	-3.15568e-05	3.15568e-05
27	0.0920915	0.0921055	1.39561e-05	1.39561e-05
23	0.0849605	0.0847904	-0.000170119	0.000170119
28	0.0806306	0.0805952	-3.54419e-05	3.54419e-05
22	0.069891	0.0699028	1.1786e-05	1.1786e-05
29	0.0668794	0.0670075	0.000128099	0.000128099
21	0.0530149	0.0530025	-1.23642e-05	1.23642e-05
30	0.0527071	0.0528001	9.30076e-05	9.30076e-05
31	0.0395713	0.0395692	-2.06349e-06	2.06349e-06
20	0.0368187	0.0368048	-1.38824e-05	1.38824e-05

Table 3: Exact and Monte Carlo probability masses at the most likely integer completion times.

t	Exact $F(t)$	MC $\hat{F}_n(t)$	Error	Absolute error	Pointwise 95% half-width	Inside DKW band
0	0	0	0	0	0	Yes
10	2.45913e-07	1e-07	-1.45913e-07	1.45913e-07	3.0736e-07	Yes
15	0.00156689	0.0015702	3.31346e-06	3.31346e-06	2.45151e-05	Yes
17	0.0111115	0.0111405	2.554e-05	2.554e-05	6.49805e-05	Yes
18	0.0242768	0.0242905	1.3695e-05	1.3695e-05	9.53927e-05	Yes
20	0.0843129	0.0843369	2.40376e-05	2.40376e-05	0.000172217	Yes
25	0.488772	0.48856	-0.000212389	0.000212389	0.000309825	Yes
30	0.880401	0.880449	4.7869e-05	4.7869e-05	0.000201122	Yes
33	0.967809	0.967841	3.19653e-05	3.19653e-05	0.0001094	Yes
35	0.988714	0.988742	2.77305e-05	2.77305e-05	6.54724e-05	Yes
40	0.999529	0.999537	8.50669e-06	8.50669e-06	1.34497e-05	Yes

Table 5: Monte Carlo validation diagnostics against the distribution.

Quantity	Value
MC mean $\hat{\mathbb{E}}[T_{\text{project}}]$	25.784
MC variance	15.7192
MC standard deviation	3.96474
MC median	26
MC 75% quantile	28
MC 90% quantile	31
MC 95% quantile	33
MC 99% quantile	36

The diagnostics above show that the Monte Carlo simulation gives a very close approximation to the exact discrete completion-time distribution in this validation example. Since the exact shifted-Poisson CDF can still be computed for this relatively small project network, it provides a useful benchmark for checking that the simulation logic is correct. The empirical CDF, PMF, and survival-tail estimates all closely match the exact values, and the largest CDF error remains

well within the DKW reference band.

The runtime comparison also illustrates why the exact method is mainly useful as a validation tool. In this example, the exact CDF calculation is roughly 960 times slower than the vectorized Monte Carlo simulation. The exact method evaluates the distribution threshold by threshold, while the Monte Carlo method generates one sample of project completion times and then uses that same sample to estimate the full distribution. This makes Monte Carlo much more practical for larger project networks and for richer stochastic risk models where an exact calculation is no longer feasible.

Another advantage of the Monte Carlo approach is that the same simulated sample can be used to estimate many project-risk quantities directly. Once the simulated project completion times

$$T_{\text{Project}}^{(1)}, \dots, T_{\text{Project}}^{(M)}$$

have been generated, the mean, variance, standard deviation, quantiles, tail probabilities, and delay penalties can all be estimated from the same output. For example,

$$\hat{\mathbb{E}}[T_{\text{Project}}] = \frac{1}{M} \sum_{s=1}^M T_{\text{Project}}^{(s)}.$$

This makes the simulation output easy to interpret from a project-management perspective, since the same set of simulated project outcomes can be used to answer several practical questions about expected duration, schedule uncertainty, and overrun risk.

It is also worth noting that the shifted-Poisson duration model used in this experiment is unbounded. Therefore, even though it is light-tailed, it can theoretically generate very large activity durations. This is different from bounded duration models, such as PERT-type models, where each activity duration is restricted to a finite interval. The fact that the Monte Carlo approximation performs well in this unbounded validation case gives useful evidence that the implementation is behaving correctly. For bounded duration distributions, estimates of means, CDF values, and moderate quantiles are often more stable because extreme realizations are limited by construction. However, very high tail quantiles can still require large sample sizes whenever the relevant event has small probability.

4 Practical Full Cost and Time Simulation with Risk

4.1 Purpose of the Full Project Example

We now apply the simulation framework to a larger project example with both schedule risk and cost risk. The purpose of this section is to show how the framework can be used in a setting that is closer to a standard project-risk analysis workflow. In practical terms, this means starting from a project schedule, activity cost data, and a risk register, and then using Monte Carlo simulation to estimate how uncertainty affects project duration and project cost.

The example also serves as a diagnostic test of the implementation. Before interpreting the simulation results from a project-management perspective, we check that the simulated values agree with the analytical expected values implied by the model. This is important because a Monte Carlo simulation can produce realistic-looking outputs even when the underlying implementation is wrong. The expected-value checks therefore provide a basic quality-control step before using the results for risk interpretation.

4.2 Reference Project and Connection to Project Risk Practice

The reference project is based on the case study and supplementary Excel workbook from Acebes et al.¹⁶. Their paper is motivated by a common project-management problem: probability-impact matrices are easy to use, but they often do not show how much each risk affects the actual project objectives. Instead of only classifying risks as low, medium, or high, Acebes et al. use Monte Carlo simulation to estimate the quantitative impact of each risk on project duration and project cost.

This is closely related to the aim of this report. The goal is not only to assign risk scores, but to simulate how the project can unfold under uncertainty. In this sense, the simulation framework can be viewed as a quantitative risk assessment layer built on top of the project schedule, work-package cost model, and risk register.

The case study in Acebes et al. contains an engineering, procurement, and construction project with 32 activities, including fictitious initial and final activities. The original activity durations are modelled using triangular distributions with minimum, most likely, and maximum values. Costs are divided into fixed costs and variable costs, where the variable cost depends on activity duration. Under the planned deterministic schedule, the project duration is 300 days and the planned cost is 30000 in thousands of monetary units.¹⁷

This planned duration and planned cost are useful reference values for interpreting the simulation results. Since the present implementation includes uncertainty and risk effects, the simulated expected duration and cost should not be expected to equal the planned values exactly. However, they should remain in a reasonable range relative to the original project scale. This gives a practical sanity check: the model should produce outputs that are compatible with the size and structure of the reference project.

¹⁶Acebes et al., see n. 2.

¹⁷Acebes et al., see n. 2.

4.3 How the Data Is Translated into the Simulation Framework

There are important differences between the simulation in Acebes et al. and the implementation used here. We use their project as a reference case, but the data is translated into the notation and discrete-time framework developed in this report.

The original supplementary workbook stores activity data, precedence relations, risk probabilities, and risk impacts in a spreadsheet format designed for their simulation tool. In the cleaned workbook used here, the data is reorganized into two explicit tables:

1. an activity table, containing activity IDs, predecessors, duration parameters, fixed costs, variable costs, and connected risks;
2. a risk table, containing risk IDs, affected activities, probability ranges, impact ranges, and whether the risk affects schedule or cost.

This restructuring is useful from a project-management perspective because it separates the schedule model from the risk register. The activity table describes the project plan, while the risk table describes uncertain events that may affect that plan. It is also useful from a technical perspective because each modeling assumption becomes explicit before simulation.

The cleaned workbook contains 32 activities, 9 schedule risks, and 6 cost risks. The original workbook uses 20000 Monte Carlo simulations and reports reference values that can be used for comparison:

$$q_{0.95}^{\text{ref}}(T_{\text{Project}}) = 323.433943, \quad q_{0.95}^{\text{ref}}(C_{\text{Project}}) = 30339.316006.$$

These values are useful reference points, but they should not be treated as exact validation targets unless the same distributional assumptions and the same risk-isolation procedure are used. In particular, the current implementation rewrites the original activity duration model into the distributional framework used in this report. Therefore, the most important validation in this section is the internal consistency check between analytical expected values and Monte Carlo sample means.

A second point where care is needed is risk *R9*. In the descriptive risk list, *R9* is described as a medium schedule impact. However, the computational data in the workbook uses a smaller implemented impact interval. The updated workbook follows the computational input used by the spreadsheet rather than only the descriptive label. Therefore, the simulation should be interpreted as reproducing the computational structure of the workbook, not every qualitative label in the article text.

4.4 Implemented Schedule and Cost Model

For each work package $i \in \mathcal{W}$, let Z_i denote the baseline activity duration before schedule-risk effects are added. Schedule risks are modelled as additive duration effects. If work package i

has r_i^t schedule risks, then the full simulated duration is

$$D_i = Z_i + \sum_{j=1}^{r_i^t} R_i^{t,j}.$$

Each schedule-risk effect is written as

$$R_i^{t,j} = \xi_i^{t,j} U_i^{t,j},$$

where $U_i^{t,j}$ is the sampled schedule impact and

$$\xi_i^{t,j} \mid P_i^{t,j} \sim \text{Bernoulli}(P_i^{t,j}).$$

Here $P_i^{t,j}$ is the sampled probability of occurrence. This means that the model separates two pieces of risk information that project managers normally record in a risk register: the probability that the risk occurs and the impact if it occurs.

The baseline cost of work package i is denoted by A_i . In the implementation, this baseline cost contains a duration-dependent part and a fixed-cost part. If K_i is the variable daily cost and $H_i^1, \dots, H_i^{h_i}$ are the fixed-cost terms, then

$$A_i = K_i D_i + \sum_{j=1}^{h_i} H_i^j.$$

Cost risks are then added to the baseline cost. If work package i has r_i^c cost risks, then

$$C_i = A_i + \sum_{j=1}^{r_i^c} R_i^{c,j},$$

where

$$R_i^{c,j} = \xi_i^{c,j} U_i^{c,j}, \quad \xi_i^{c,j} \mid P_i^{c,j} \sim \text{Bernoulli}(P_i^{c,j}).$$

The total project cost is therefore

$$C_{\text{Project}} = \sum_{i \in \mathcal{W}} C_i.$$

This structure is close to the project-risk logic used by Acebes et al., where activity durations are uncertain, activity cost contains fixed and duration-dependent components, and risks affect either duration or cost. The main difference is that the present report makes the probability model and the simulation assumptions explicit in mathematical notation before implementing them in code.

4.5 Expected-Value Checks for the Simulation

The first diagnostic is an expected-value check. This is useful because several quantities in the simulation have analytical expected values. Under the independence assumptions used in the

implementation,

$$\mathbb{E}[R_i^{t,j}] = \mathbb{E}[P_i^{t,j}]\mathbb{E}[U_i^{t,j}], \quad \mathbb{E}[R_i^{c,j}] = \mathbb{E}[P_i^{c,j}]\mathbb{E}[U_i^{c,j}].$$

Therefore,

$$\mathbb{E}[D_i] = \mathbb{E}[Z_i] + \sum_{j=1}^{r_i^t} \mathbb{E}[R_i^{t,j}],$$

and

$$\mathbb{E}[C_i] = \mathbb{E}[A_i] + \sum_{j=1}^{r_i^c} \mathbb{E}[R_i^{c,j}].$$

By linearity of expectation, the project-level expected cost is

$$\mathbb{E}[C_{\text{Project}}] = \sum_{i \in \mathcal{W}} \mathbb{E}[C_i].$$

From a project-management perspective, this check answers a simple but important question: does the simulation reproduce the average duration and cost values implied by the input assumptions? If the Monte Carlo sample means are close to the analytical expected values, then the simulation is behaving consistently with the model assumptions.

Table 6: Analytic expected values and Monte Carlo means by work package.

Act.	$\mathbb{E}[D_i]$	$\bar{D}_i$	e_D	$\mathbb{E}[C_i]$	$\bar{C}_i$	e_C
A1	15.000	14.999	-0.006%	15.000	15.000	-0.003%
A2	44.000	44.005	0.012%	280.000	279.924	-0.027%
A3	30.000	30.002	0.008%	135.000	135.009	0.007%
A4	55.000	54.999	-0.002%	247.500	247.482	-0.007%
A5	70.000	70.000	0.000%	525.000	525.037	0.007%
A6	90.188	90.188	0.001%	315.656	315.678	0.007%
A7	70.000	70.001	0.002%	700.000	700.023	0.003%
A8	25.000	24.999	-0.002%	756.125	756.515	0.052%
A9	85.000	84.999	-0.002%	4,200.000	4,200.005	0.000%
A10	85.000	85.001	0.002%	3,675.000	3,675.213	0.006%
A11	80.000	80.002	0.002%	7,000.000	6,999.984	-0.000%
A12	112.500	112.502	0.002%	75.000	74.995	-0.007%
A13	84.062	84.050	-0.014%	1,738.125	1,738.008	-0.007%
A14	30.000	29.999	-0.002%	280.000	279.999	-0.001%
A15	15.000	15.000	0.002%	90.000	89.999	-0.001%
A16	190.000	189.998	-0.001%	642.000	642.002	0.000%
A17	55.000	54.999	-0.001%	564.000	564.000	-0.000%
A18	60.000	60.000	-0.001%	874.000	873.992	-0.001%
A19	55.000	55.002	0.004%	905.000	905.015	0.002%
A20	102.000	101.988	-0.012%	1,225.800	1,225.769	-0.003%
A21	25.000	25.000	0.000%	365.000	365.002	0.000%
A22	20.000	20.000	-0.001%	347.000	346.990	-0.003%
A23	40.812	40.813	0.001%	482.900	482.904	0.001%
A24	40.000	40.000	0.001%	395.000	394.986	-0.004%
A25	45.000	45.000	-0.000%	435.000	435.005	0.001%
A26	85.812	85.813	0.000%	1,054.425	1,054.600	0.017%
A27	80.000	79.999	-0.001%	740.000	740.035	0.005%
A28	67.000	67.000	0.001%	2,050.000	2,050.075	0.004%
A29	15.000	15.000	0.003%	30.000	30.001	0.004%
A30	60.525	60.526	0.002%	499.425	499.432	0.001%
A31	15.000	15.000	0.002%	321.250	321.069	-0.056%
A32	15.000	15.000	0.001%	45.000	45.000	-0.001%

The largest deviations between analytical expected values and Monte Carlo means are reported separately. This table is useful because it focuses attention on the activities where the simulation differs most from its expected-value benchmark.

Table 7: Largest relative deviations between analytic expected values and Monte Carlo means.

Act.	$\mathbb{E}[D_i]$	$\bar{D}_i$	e_D	$\mathbb{E}[C_i]$	$\bar{C}_i$	e_C	$\max e $
A31	15.000	15.000	0.002%	321.250	321.069	-0.056%	0.056%
A8	25.000	24.999	-0.002%	756.125	756.515	0.052%	0.052%
A2	44.000	44.005	0.012%	280.000	279.924	-0.027%	0.027%
A26	85.812	85.813	0.000%	1,054.425	1,054.600	0.017%	0.017%
A13	84.062	84.050	-0.014%	1,738.125	1,738.008	-0.007%	0.014%
A20	102.000	101.988	-0.012%	1,225.800	1,225.769	-0.003%	0.012%
A3	30.000	30.002	0.008%	135.000	135.009	0.007%	0.008%
A4	55.000	54.999	-0.002%	247.500	247.482	-0.007%	0.007%

4.6 Risk-Level Diagnostics and Risk Prioritisation

The same expected-value check is also performed at the risk level. For each risk, the simulation records the sampled occurrence probability, the sampled severity, the occurrence indicator, and the realized effect. This is important because each risk has two layers of randomness: first the probability of occurrence is sampled, and then the occurrence indicator is sampled conditionally on that probability.

Table 8: Simulated risk effects compared with stored expected effects. Cost values are measured in 10^3 monetary units.

Risk	Obj.	Act.	$\bar{\xi}$	$\bar{U}$	$\bar{R}$	$\mathbb{E}[R]$	$ \bar{R} - \mathbb{E}[R] $	Unit
R15	Cost	A31	6.50%	4,249.388	276.068	276.250	0.182	10^3 MU
R10	Cost	A2	39.97%	425.103	169.923	170.000	0.077	10^3 MU
R11	Cost	A8	6.52%	1,625.366	106.009	105.625	0.384	10^3 MU
R13	Cost	A26	6.51%	1,625.160	105.778	105.625	0.153	10^3 MU
R12	Cost	A13	6.50%	425.124	27.652	27.625	0.027	10^3 MU
R14	Cost	A30	1.49%	49.996	0.745	0.750	0.005	10^3 MU
R2	Time	A2	40.01%	34.998	14.005	14.000	0.005	days
R8	Time	A28	20.01%	34.997	7.002	7.000	0.002	days
R5	Time	A20	19.96%	35.000	6.988	7.000	0.012	days
R3	Time	A13	6.48%	62.497	4.050	4.062	0.012	days
R4	Time	A12	20.02%	12.497	2.501	2.500	0.001	days
R6	Time	A26	6.50%	12.501	0.812	0.812	0.000	days
R9	Time	A23	6.49%	12.500	0.812	0.812	0.001	days
R7	Time	A30	1.50%	34.999	0.525	0.525	0.000	days
R1	Time	A6	1.50%	12.502	0.188	0.188	0.001	days

The risks with the largest simulated effects are shown in the next table.

Table 9: Largest simulated risk effects by risk type. Cost values are measured in 10^3 monetary units.

Risk	Obj.	Act.	$\bar{\xi}$	$\bar{U}$	$\bar{R}$	$\mathbb{E}[R]$	$ \bar{R} - \mathbb{E}[R] $	Unit
R15	Cost	A31	6.50%	4,249.388	276.068	276.250	0.182	10^3 MU
R10	Cost	A2	39.97%	425.103	169.923	170.000	0.077	10^3 MU
R11	Cost	A8	6.52%	1,625.366	106.009	105.625	0.384	10^3 MU
R13	Cost	A26	6.51%	1,625.160	105.778	105.625	0.153	10^3 MU
R12	Cost	A13	6.50%	425.124	27.652	27.625	0.027	10^3 MU
R2	Time	A2	40.01%	34.998	14.005	14.000	0.005	days
R8	Time	A28	20.01%	34.997	7.002	7.000	0.002	days
R5	Time	A20	19.96%	35.000	6.988	7.000	0.012	days
R3	Time	A13	6.48%	62.497	4.050	4.062	0.012	days
R4	Time	A12	20.02%	12.497	2.501	2.500	0.001	days

This is the part of the analysis that is closest to the motivation in Acebes et al. Their method aims to move beyond qualitative probability-impact matrices by ranking risks according to

their simulated impact on project objectives. The present implementation follows the same general project-risk logic, but it also adds explicit expected-value diagnostics. This makes the simulation more transparent because the user can check not only which risks appear important, but also whether the sampled risk effects agree with the mathematical assumptions used by the model.

The original workbook also contains a risk-ranking sheet. The largest reference schedule effects in that sheet are approximately *R3*, *R5*, *R2*, and *R8*, while the largest reference cost effects are approximately *R15*, *R13*, *R11*, and *R5*. These are useful directional checks. However, they should not be used as exact row-by-row validation targets unless the same simulation model and the same risk-isolation procedure are used. The tables above instead validate and interpret the model that is actually implemented in this report.

4.7 Project-Level Duration and Cost Diagnostics

After checking the work-package and risk-level outputs, we now check the results at the full project level. This is the most important level for practical project management, because it gives the simulated project completion time and total project cost.

For each Monte Carlo replication $s = 1, \dots, M$, the simulation produces one project completion time

$$T_{\text{Project}}^{(s)}$$

and one total project cost

$$C_{\text{Project}}^{(s)} = \sum_{i \in \mathcal{W}} C_i^{(s)}.$$

The Monte Carlo estimates are then calculated directly from these samples.

For cost, we can also compare the Monte Carlo mean with the analytical project-cost mean implied by the model. In the implementation, this value is computed by calling

`project.expected_project_cost()`.

This gives a useful diagnostic: if the Monte Carlo mean cost is close to the analytical expected cost, then the cost simulation is consistent with the model assumptions.

Table 10: Project-level duration and cost diagnostics.

Quantity	Analytical mean	MC mean	MC error	P50	P90	P95
Project completion time (days)	–	334.1462	–	333.0000	369.0000	378.0000
Total project cost (monetary units)	31,008.2062	31,008.7462	0.5400	30,754.3412	32,935.4263	34,215.3949

The project-cost diagnostic is especially strong. The analytical expected cost is 31,008.2062, while the Monte Carlo mean is 31,008.7462. The difference is only 0.5400 monetary units, which is negligible relative to the total project cost. This gives a clear check that the cost simulation is reproducing the expected value implied by the model.

For project duration, we do not report an analytical expected value. This is because the project completion time is defined through a maximum over the network, and in general

$$\mathbb{E}[T_{\text{Project}}]$$

does not have the same simple additive structure as project cost. The duration results are therefore interpreted directly from the simulation output. In this run, the mean project completion time is about 334.15 days, with median 333 days, P90 equal to 369 days, and P95 equal to 378 days.

To check the stability of the high-percentile estimates, the full simulation sample is split into blocks of 500000 replications. The P90 and P95 values are then recomputed within each block for both duration and cost.

Table 11: Block diagnostics for project-level P90 and P95 estimates.

Quantity	Quantile	Full sample	Block mean	Block SD	95% half-width
Project completion time (days)	P90	369.0000	369.0000	0.0000	0.0000
Project completion time (days)	P95	378.0000	378.0000	0.0000	0.0000
Total project cost (monetary units)	P90	32,935.4263	32,935.5926	5.8417	3.6207
Total project cost (monetary units)	P95	34,215.3949	34,215.3263	10.2593	6.3588

The block diagnostics show that the P90 and P95 estimates are stable at this simulation size. For duration, all blocks give the same integer-valued P90 and P95. This does not mean that there is literally no uncertainty in the estimator, but it does show that the estimated duration quantiles are stable at the day-level resolution used here.

For cost, the block variation is also small. The P90 cost has a block standard deviation of about 5.84 monetary units, and the P95 cost has a block standard deviation of about 10.26 monetary units. Compared with project costs around 31,000 to 34,000, this is very small. The block check therefore suggests that the reported cost quantiles are numerically stable for the chosen sample size.

Together, these project-level diagnostics support the simulation results. The cost mean agrees with the analytical expected value, and the high-percentile estimates are stable across large independent blocks of the simulation. This gives a stronger basis for interpreting the later activity-level exposure plot.

4.8 Activity-Level Cost and Schedule Exposure

The previous diagnostics summarized the simulation at the project level. We now move one level down and study the exposure of individual work packages. This is useful from a project-management perspective because project managers usually do not only need to know the expected project duration or total cost. They also need to know which work packages deserve attention during planning, monitoring, and risk response.

This subsection introduces an activity-level exposure metric that combines two familiar project-

control questions:

1. Which work packages consume a large share of the total project cost?
2. Which work packages contribute a large share of project duration when they are critical?

This type of diagnostic is related to the schedule-risk-analysis literature, where Monte Carlo simulation is used to identify activities that are likely to influence project duration or cost.¹⁸ Traditional activity sensitivity measures include the Criticality Index, Significance Index, Schedule Sensitivity Index, and Cruciality Index. These measures are useful, but they usually focus on schedule sensitivity alone, or on correlation with project duration or cost. The metric introduced here is different because it places cost exposure and critical-time exposure in the same plot.

This is also connected to recent work on quantitative project-risk prioritisation. Acebes et al. argue that project risks should be evaluated in terms of their quantitative impact on project duration and project cost, rather than only through qualitative probability-impact matrices.¹⁹ The metric below follows the same practical motivation, but applies it at the work-package level instead of only at the risk-event level.

4.8.1 Definition of the Cost and Critical-Time Exposure Metric

For each Monte Carlo sample $s = 1, \dots, M$, let $D_i^{(s)}$ be the simulated duration of work package i , and let $C_i^{(s)}$ be the simulated cost of work package i . Let

$$T_{\text{Project}}^{(s)}$$

be the simulated project completion time, and let

$$C_{\text{Project}}^{(s)} = \sum_{i \in \mathcal{W}} C_i^{(s)}$$

be the simulated total project cost. Let $\mathcal{C}^{(s)}$ denote the set of work packages that lie on at least one critical path in simulation sample s .

The first quantity is the samplewise cost ratio of work package i :

$$Y_i^{(s)} = \frac{C_i^{(s)}}{C_{\text{Project}}^{(s)}}.$$

This is the share of total project cost taken up by work package i in sample s . The empirical mean cost ratio is

$$\hat{y}_i = \frac{1}{M} \sum_{s=1}^M Y_i^{(s)} = \frac{1}{M} \sum_{s=1}^M \frac{C_i^{(s)}}{C_{\text{Project}}^{(s)}}.$$

¹⁸D. T. Hulett. “Project Schedule Risk Analysis: Monte Carlo Simulation or PERT?”. In: *PM Network* 14.2 (2000), pp. 43–47. URL: <https://www.pmi.org/learning/library/project-schedule-risk-analysis-simulation-4620>; Vanhoucke, see n. 1.

¹⁹Acebes et al., see n. 2.

The second quantity is the samplewise critical-time ratio:

$$X_i^{(s)} = \mathbf{1}_{\{i \in \mathcal{C}^{(s)}\}} \frac{D_i^{(s)}}{T_{\text{Project}}^{(s)}}.$$

If work package i is on a critical path in sample s , then $X_i^{(s)}$ records how much of the realized project completion time is taken up by that work package. If work package i is not critical in sample s , then $X_i^{(s)} = 0$.

The empirical mean critical-time ratio is

$$\hat{x}_i = \frac{1}{M} \sum_{s=1}^M X_i^{(s)} = \frac{1}{M} \sum_{s=1}^M \mathbf{1}_{\{i \in \mathcal{C}^{(s)}\}} \frac{D_i^{(s)}}{T_{\text{Project}}^{(s)}}.$$

We also define the empirical criticality probability

$$\hat{\pi}_i = \frac{1}{M} \sum_{s=1}^M \mathbf{1}_{\{i \in \mathcal{C}^{(s)}\}}.$$

The proposed activity-level metric is the pair

$$(\hat{x}_i, \hat{y}_i).$$

The horizontal axis measures mean critical-time exposure, and the vertical axis measures mean cost exposure. The color and point size show $\hat{\pi}_i$, the probability that the work package is critical. A work package in the upper-right part of the plot is therefore important in both dimensions: it consumes a relatively large share of total project cost and it also contributes a relatively large share of project duration when it is critical.

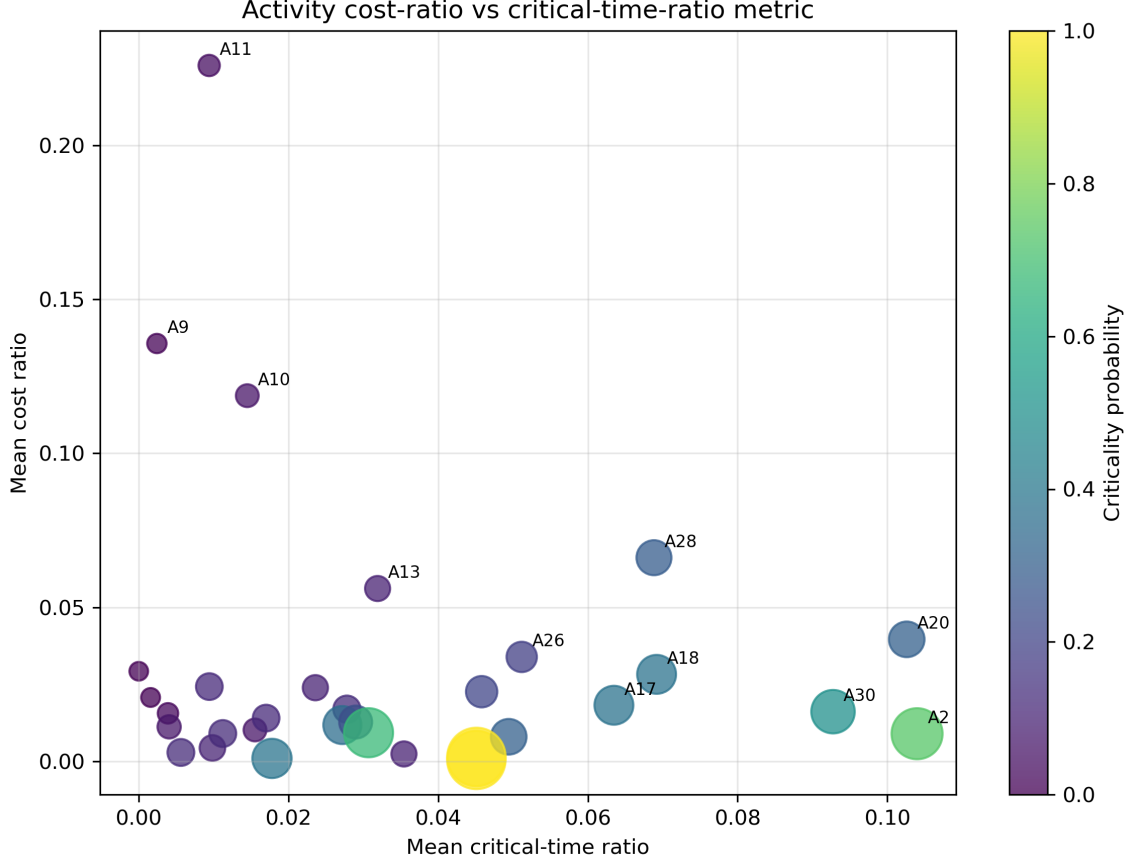


Figure 11: Activity cost-ratio and critical-time-ratio metric. The horizontal axis shows $\hat{x}_i$, the average share of project completion time taken up by work package i when it is critical, with value zero in samples where it is not critical. The vertical axis shows $\hat{y}_i$, the average share of total project cost taken up by work package i . The color and point size show the empirical criticality probability $\hat{\pi}_i$.

The work packages with the largest values of this metric are listed below.

Table 12: Activities with the largest mean critical-time ratio and their corresponding mean cost ratio.

Act.	$\hat{x}_i$	$\hat{y}_i$	$\hat{\pi}_i$	$\mathbb{E}[X_i i \in \mathcal{C}]$
A2	10.40%	0.90%	73.56%	14.13%
A20	10.26%	3.96%	30.08%	34.11%
A30	9.28%	1.61%	50.27%	18.45%
A18	6.92%	2.83%	37.99%	18.21%
A28	6.88%	6.61%	28.67%	24.01%
A17	6.35%	1.82%	37.99%	16.71%
A26	5.12%	3.39%	18.74%	27.30%
A4	4.94%	0.80%	30.08%	16.43%
A7	4.58%	2.26%	20.83%	21.99%
A32	4.51%	0.15%	100.00%	4.51%
A1	4.51%	0.05%	100.00%	4.51%
A12	3.54%	0.24%	9.25%	38.30%

4.8.2 Practical Interpretation

The metric should be interpreted as an activity-level exposure metric, not as a marginal sensitivity measure. This distinction is important. If a work package lies on a critical path, it

does not automatically mean that reducing its duration by one day will reduce the total project duration by one day. This is especially true when there are tied critical paths.

In samples with multiple critical paths, $\mathcal{C}^{(s)}$ is the union of all work packages that lie on at least one critical path. Therefore,

$$\sum_{i \in \mathcal{W}} \mathbf{1}_{\{i \in \mathcal{C}^{(s)}\}} \frac{D_i^{(s)}}{T_{\text{Project}}^{(s)}}$$

can be larger than 1. This does not make the metric wrong. It only means that the metric measures exposure to critical-path behavior, rather than the exact marginal effect of reducing one work package by one time unit.

From a project-management perspective, the plot is best used as a screening tool. It identifies work packages that combine high cost exposure with high schedule-critical exposure. These are natural candidates for closer monitoring, more careful progress reporting, or more detailed mitigation analysis.

The interpretation can be summarized as follows:

- A high value of $\hat{y}_i$ means that the work package is a large contributor to total project cost.
- A high value of $\hat{x}_i$ means that the work package contributes meaningfully to project duration in the simulations where it is critical.
- A high value of $\hat{\pi}_i$ means that the work package is frequently on a critical path.

The most important management attention should go to work packages that are high in several of these dimensions. For example, a work package with high cost exposure but low critical-time exposure may be important for cost control, but less important for schedule acceleration. A work package with high critical-time exposure but low cost exposure may be important for schedule control, but less important for cost reduction. A work package with high values in both dimensions is important for both cost and schedule performance.

Compared with a traditional probability-impact matrix, this diagnostic has the advantage that the output is expressed in project-level quantities. Cost exposure is measured as a share of total project cost, and schedule exposure is measured as a share of realized project completion time. This makes the result easier to connect to practical project-control decisions, such as which activities to monitor more closely, where to investigate mitigation options, and where cost and schedule priorities may conflict.

References

- Acebes, F. et al. “Beyond probability-impact matrices in project risk management: A quantitative methodology for risk prioritisation”. In: *Humanities and Social Sciences Communications* 11.670 (2024). DOI: 10.1057/s41599-024-03180-5. URL: <https://doi.org/10.1057/s41599-024-03180-5>.
- Bara, M. “Enhanced Monte Carlo Simulation for Project Risk Analysis: Integrating Cost and Schedule Impacts with Time-Shifted Risks and Dependency Modeling”. In: *Applied Operations and Analytics* 1.1 (2025), pp. 1–15. DOI: 10.1080/29966892.2025.2552675.
- Hochbaum, D. S. *Lecture Notes for IEOR 266: Graph Algorithms and Network Flows*. Lecture notes, Department of Industrial Engineering and Operations Research, University of California, Berkeley. Updated 2024. Accessed: 2026-05-12. 2024. URL: <https://hochbaum.ieor.berkeley.edu/files/266Notes-F2020.pdf>.
- Hulett, D. T. “Project Schedule Risk Analysis: Monte Carlo Simulation or PERT?”. In: *PM Network* 14.2 (2000), pp. 43–47. URL: <https://www.pmi.org/learning/library/project-schedule-risk-analysis-simulation-4620>.
- Lattimore, T. and C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020. ISBN: 9781108486828. DOI: 10.1017/9781108571401.
- Meucci, A. “Review of Discrete and Continuous Processes in Finance: Theory and Applications”. In: *SSRN Electronic Journal* (2009). DOI: 10.2139/ssrn.1373102. URL: <https://ssrn.com/abstract=1373102>.
- NetworkX Developers. *topological_sort*. NetworkX 3.6.1 documentation. Accessed: 2026-05-12. 2025. URL: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.dag.topological_sort.html.
- Oberemok, N. and I. Oberemok. “Stochastic Processes in Project Management”. In: *EUREKA: Social and Humanities* 6 (2018), pp. 3–9. DOI: 10.21303/2504-5571.2018.00799.
- Society for Risk Analysis. *Society for Risk Analysis Glossary*. Updated August 2018. Expert group led by Terje Aven. Society for Risk Analysis. Aug. 2018. URL: <https://www.sra.org/wp-content/uploads/2020/04/SRA-Glossary-FINAL.pdf>.
- Soltan, S. and M. Ashrafi. “Application of Reinforcement Learning for Integrating Project Risk Analysis and Risk Response Planning: A Case Study on Construction Projects”. In: *Journal of Project Management* 10.1 (2025), pp. 71–86. DOI: 10.5267/j.jpm.2024.11.001.
- Vanhoucke, M. *Schedule Risk Analysis: Measuring the Time Sensitivity of an Activity*. 2011. URL: https://www.pmknowledgecenter.com/dynamic_scheduling/risk/schedule-risk-analysis-measuring-time-sensitivity-activity (visited on 05/19/2026).

A Useful Theorems for Analytical Expected-Value Checks

The following result is useful for diagnosing the Monte Carlo simulation. It gives an analytical expected-value expression that the simulated sample mean can be compared against. This is especially useful for cost, because total project cost is built by adding work-package costs and project-level costs.

Theorem A.1 (Expected project cost). *Assume that the work-package costs and the project-level cost O have finite expected values. Then*

$$\mathbb{E}[C_{\text{Project}}] = \sum_{i \in \mathcal{W}} \sum_{m > n} \mathbb{P}(T_i = m, S_i = n) \mathbb{E}[C_i((t)_{n < t \leq m}) \mid T_i = m, S_i = n] + \mathbb{E}[O].$$

Here, terms with $\mathbb{P}(T_i = m, S_i = n) = 0$ are omitted.

Proof. The total project cost is written as

$$C_{\text{Project}} = C_{WP} + O,$$

where

$$C_{WP} = \sum_{i \in \mathcal{W}} C_i((t)_{S_i < t \leq T_i}).$$

By linearity of expectation,

$$\mathbb{E}[C_{\text{Project}}] = \sum_{i \in \mathcal{W}} \mathbb{E}[C_i((t)_{S_i < t \leq T_i})] + \mathbb{E}[O].$$

It remains to rewrite the expected cost of one work package. Since S_i and T_i are integer-valued, exactly one of the events

$$\{T_i = m, S_i = n\}$$

occurs. Therefore,

$$C_i((t)_{S_i < t \leq T_i}) = \sum_{m > n} \mathbf{1}_{\{T_i = m, S_i = n\}} C_i((t)_{n < t \leq m}) \quad \text{a.s.}$$

Taking expectations gives

$$\mathbb{E}[C_i((t)_{S_i < t \leq T_i})] = \sum_{m > n} \mathbb{E}[\mathbf{1}_{\{T_i = m, S_i = n\}} C_i((t)_{n < t \leq m})].$$

For any event A with $\mathbb{P}(A) > 0$, we use the elementary identity

$$\mathbb{E}[X \mathbf{1}_A] = \mathbb{P}(A) \mathbb{E}[X \mid A].$$

Applying this with

$$A = \{T_i = m, S_i = n\}$$

and

$$X = C_i((t)_{n < t \leq m}),$$

we obtain

$$\mathbb{E} [C_i((t)_{S_i < t \leq T_i})] = \sum_{m > n} \mathbb{P}(T_i = m, S_i = n) \mathbb{E} [C_i((t)_{n < t \leq m}) \mid T_i = m, S_i = n].$$

Summing this expression over all work packages and adding $\mathbb{E}[O]$ gives

$$\mathbb{E}[C_{\text{Project}}] = \sum_{i \in \mathcal{W}} \sum_{m > n} \mathbb{P}(T_i = m, S_i = n) \mathbb{E} [C_i((t)_{n < t \leq m}) \mid T_i = m, S_i = n] + \mathbb{E}[O].$$

□

This result does not require the work-package costs or durations to be independent. Independence is only needed later if we want to simplify the model further, simulate variables separately, or study the full distribution of total project cost. For the expected total cost, the key property is linearity of expectation.

Theorem A.2. Let $\mathcal{P}$ be the set of all complete paths in the project DAG, and assume that the work package durations $D_1, \dots, D_m$ are independent nonnegative integer-valued random variables. For each path $p_\theta \in \mathcal{P}$, let

$$D(p_\theta) = \sum_{i \in p_\theta} D_i.$$

Then the project completion time is

$$T_{\text{project}} = \max_{p_\theta \in \mathcal{P}} D(p_\theta).$$

Let

$$\mathcal{N} = \{i : \text{the work package } i \text{ belongs to at least two paths in } \mathcal{P}\}$$

be the set of shared work packages, and define

$$\mathcal{U}_\theta = p_\theta \setminus \mathcal{N}.$$

Then, for every $t \geq 0$,

$$\mathbb{P}(T_{\text{project}} \leq t) = \sum_{(d_i)_{i \in \mathcal{N}} \in \mathbb{N}_0^{|\mathcal{N}|}} \left(\prod_{i \in \mathcal{N}} \mathbb{P}(D_i = d_i) \right) \prod_{p_\theta \in \mathcal{P}} \mathbb{P} \left(\sum_{k \in \mathcal{U}_\theta} D_k \leq t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i \right).$$

Proof. We begin by identifying the most basic case, and then use the same idea to prove the general theorem. The first case is when each path is totally separated from every other path. This means that $|\mathcal{N}| = 0$. Then no work package is shared by two different paths, and therefore

$$\mathbb{P}(T_{\text{project}} \leq t) = \mathbb{P} \left(\bigcap_{p_\theta \in \mathcal{P}} \left\{ \sum_{k \in p_\theta} D_k \leq t \right\} \right).$$

Since the paths are separated, the collections of random variables appearing in different path sums are disjoint. Since the work package durations are mutually independent, the path sums are also mutually independent. Thus

$$\mathbb{P}(T_{\text{project}} \leq t) = \prod_{p_\theta \in \mathcal{P}} \mathbb{P} \left(\sum_{k \in p_\theta} D_k \leq t \right).$$

This means that if we can rewrite the general case in a way where the remaining path sums depend on disjoint collections of random variables, then we can use the same independence reasoning.

Now consider the general case where $\mathcal{N}$ may be nonempty. Write

$$\mathcal{N} = \{i_1, \dots, i_n\}.$$

Fix a vector of possible values

$$(d_i)_{i \in \mathcal{N}} \in \mathbb{N}_0^{|\mathcal{N}|}.$$

On the event

$$\bigcap_{i \in \mathcal{N}} \{D_i = d_i\},$$

we have, for every path $p_\theta \in \mathcal{P}$,

$$\sum_{k \in p_\theta} D_k = \sum_{k \in p_\theta \setminus \mathcal{N}} D_k + \sum_{i \in p_\theta \cap \mathcal{N}} d_i.$$

Therefore, on this event,

$$\sum_{k \in p_\theta} D_k \leq t$$

is equivalent to

$$\sum_{k \in p_\theta \setminus \mathcal{N}} D_k \leq t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i.$$

Hence

$$\begin{aligned} & \mathbb{P}\left(\bigcap_{p_\theta \in \mathcal{P}} \left\{ \sum_{k \in p_\theta} D_k \leq t \right\} \cap \bigcap_{i \in \mathcal{N}} \{D_i = d_i\}\right) \\ &= \mathbb{P}\left(\bigcap_{p_\theta \in \mathcal{P}} \left\{ \sum_{k \in p_\theta \setminus \mathcal{N}} D_k \leq t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i \right\} \cap \bigcap_{i \in \mathcal{N}} \{D_i = d_i\}\right). \end{aligned}$$

We now observe that the sets $p_\theta \setminus \mathcal{N}$ are disjoint for different paths. Indeed, if a work package appeared in $p_\theta \setminus \mathcal{N}$ and also in $p_{\theta'} \setminus \mathcal{N}$ for $\theta \neq \theta'$, then it would belong to more than one path and would therefore be in $\mathcal{N}$, which is a contradiction.

Thus, after the values of the shared work packages have been fixed, the remaining path sums depend on disjoint collections of independent random variables. Also, these remaining path sums depend on random variables which are disjoint from the shared variables $(D_i)_{i \in \mathcal{N}}$. Therefore, by independence,

$$\begin{aligned} & \mathbb{P}\left(\bigcap_{p_\theta \in \mathcal{P}} \left\{ \sum_{k \in p_\theta \setminus \mathcal{N}} D_k \leq t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i \right\} \cap \bigcap_{i \in \mathcal{N}} \{D_i = d_i\}\right) \\ &= \left(\prod_{i \in \mathcal{N}} \mathbb{P}(D_i = d_i) \right) \prod_{p_\theta \in \mathcal{P}} \mathbb{P}\left(\sum_{k \in p_\theta \setminus \mathcal{N}} D_k \leq t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i \right). \end{aligned}$$

We now decompose the event $\{T_{\text{project}} \leq t\}$ according to the possible values of the shared durations. Since

$$T_{\text{project}} = \max_{p_\theta \in \mathcal{P}} \sum_{k \in p_\theta} D_k,$$

we have

$$\{T_{\text{project}} \leq t\} = \bigcap_{p_\theta \in \mathcal{P}} \left\{ \sum_{k \in p_\theta} D_k \leq t \right\}.$$

Thus

$$\mathbb{P}(T_{\text{project}} \leq t) = \mathbb{P}\left(\bigcup_{(d_i)_{i \in \mathcal{N}} \in \mathbb{N}_0^{|\mathcal{N}|}} \left[\bigcap_{p_\theta \in \mathcal{P}} \left\{ \sum_{k \in p_\theta \setminus \mathcal{N}} D_k \leq t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i \right\} \cap \bigcap_{i \in \mathcal{N}} \{D_i = d_i\} \right] \right).$$

The events inside this union are pairwise disjoint. Indeed, if

$$(d_i)_{i \in \mathcal{N}} \neq (d'_i)_{i \in \mathcal{N}},$$

then there exists some $j \in \mathcal{N}$ such that $d_j \neq d'_j$. The corresponding two events then contain $\{D_j = d_j\}$ and $\{D_j = d'_j\}$, which cannot both occur. Therefore, by countable additivity,

$$\mathbb{P}(T_{\text{project}} \leq t) = \sum_{(d_i)_{i \in \mathcal{N}} \in \mathbb{N}_0^{|\mathcal{N}|}} \mathbb{P}\left(\bigcap_{p_\theta \in \mathcal{P}} \left\{ \sum_{k \in p_\theta \setminus \mathcal{N}} D_k \leq t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i \right\} \cap \bigcap_{i \in \mathcal{N}} \{D_i = d_i\}\right).$$

Using the independence factorization above, we conclude that

$$\mathbb{P}(T_{\text{project}} \leq t) = \sum_{(d_i)_{i \in \mathcal{N}} \in \mathbb{N}_0^{|\mathcal{N}|}} \left(\prod_{i \in \mathcal{N}} \mathbb{P}(D_i = d_i) \right) \prod_{p_\theta \in \mathcal{P}} \mathbb{P}\left(\sum_{k \in p_\theta \setminus \mathcal{N}} D_k \leq t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i \right).$$

Since $\mathcal{U}_\theta = p_\theta \setminus \mathcal{N}$, this is exactly

$$\mathbb{P}(T_{\text{project}} \leq t) = \sum_{(d_i)_{i \in \mathcal{N}} \in \mathbb{N}_0^{|\mathcal{N}|}} \left(\prod_{i \in \mathcal{N}} \mathbb{P}(D_i = d_i) \right) \prod_{p_\theta \in \mathcal{P}} \mathbb{P}\left(\sum_{k \in \mathcal{U}_\theta} D_k \leq t - \sum_{i \in p_\theta \cap \mathcal{N}} d_i \right).$$

This proof is intentionally performed without using conditional independence as a separate theorem, since we instead use the more fundamental fact that disjoint collections of mutually independent random variables are independent. $\square$

B Mathematical niceties

These are observations that might be useful in future work but mostly here as a mathematical curiosity and mostly serves as a way for me on how to think about the modeling and serve as inspiration for future work. Especially if one wants to apply Reinforcement Learning.

A fundamental requirement is that the model should not need information from the future to determine whether an activity has started and finished after a given duration. If activity i starts at time m and has duration k , then the activity finishes at time $m + k$. Therefore, the event

$$\{S_i = m\} \cap \{D_i = k\}$$

should be observable using the information available up to time $m+k$. In mathematical notation, this means that

$$\{S_i = m\} \cap \{D_i = k\} \in \mathcal{F}_{m+k}.$$

The intuition is that we can only confirm this exact outcome after enough time has passed. First, we need to know that the activity started at time m . Then we need to observe that it lasted exactly k time units. This information is available by time $m + k$, when the activity has finished. In general, we will not require this event to be measurable with respect to $\mathcal{F}_t$ for $t < m + k$, since that would mean that the model can determine the exact duration before the activity has had enough time to finish. This condition does not primarily reflect uncertainty about the duration, but rather the information structure of the model. Even if an activity has a deterministic duration, the model should only require the event to be decidable once the relevant time has been reached. In other words, the event should be measurable with respect to the information available at time $m + k$, because this is the time at which we can determine whether the activity started at m and lasted exactly k time units. We then observe an important theorem

Theorem B.1. *Each end time T_i for each activity is a stopping time*

Proof. We remember that from the definition of a stopping time is that if $\{T_i = n\} \in \mathcal{F}_n$. This holds since $T_i = S_i + D_i$ which means that we express the event as union of S_i and D_i events.

$$\{T_i = n\} = \bigcup_{m=0}^n \{S_i = m\} \cap \{D_i = n - m\} \in \mathcal{F}_n$$

Thus it is a stopping time. □

The following theorem follows almost immediate.

Theorem B.2. *Each end time $T_{Project}$ for each activity is a stopping time*

Proof. We remember that from the definition of a stopping time is that if $\{T_{Project} = n\} \in \mathcal{F}_n$.

We then observe that we can rewrite

$$\{T_{Project} = n\} = \{\max_{i \in \mathcal{W}} T_i = n\} = \bigcup_{i \in \mathcal{W}} \left(\{T_i = n\} \cap \bigcap_{j \in \mathcal{W} \setminus \{i\}} \{T_j \leq n\} \right)$$

Then we know $\{T_i \leq n\} \in \mathcal{F}_n$ and $\{T_i = n\} \in \mathcal{F}_n$. We also have that any finite intersection and union are in $\mathcal{F}_n$. Thus we get that

$$\{T_{Project} = n\} \in \mathcal{F}_n$$

Thus it is a stopping time. □

Theorem B.3. *Each start time S_i for each activity is a stopping time.*

Proof. We first observe that $S_1 = 0$ which clearly is a stopping time. Then we remember that

$$S_i = \max_{j \in \text{Pred}(i)} (T_j + L_{i,j})$$

Which means that

$$\{S_i = n\} = \bigcup_{j \in \text{Pred}(i)} \left(\{T_j + L_{i,j} = n\} \cap \bigcap_{k \in \mathcal{W} \setminus \{j\}} \{T_k + L_k \leq n\} \right)$$

We use the same reasoning before that we can only have that for $S_i + D_i$ which means that $\{T_j + L_{i,j} = n\} \in \mathcal{F}_n$ and thus we easily conclude that

$$\{S_i = n\} \in \mathcal{F}_n$$

What this says is that we can't be sure if a workproject can begin the next if we don't now that it the other work packages plus lag it doesn't exceed the n . □